# DEBRE BERHAN UNIVERSTITY

## College of Computing

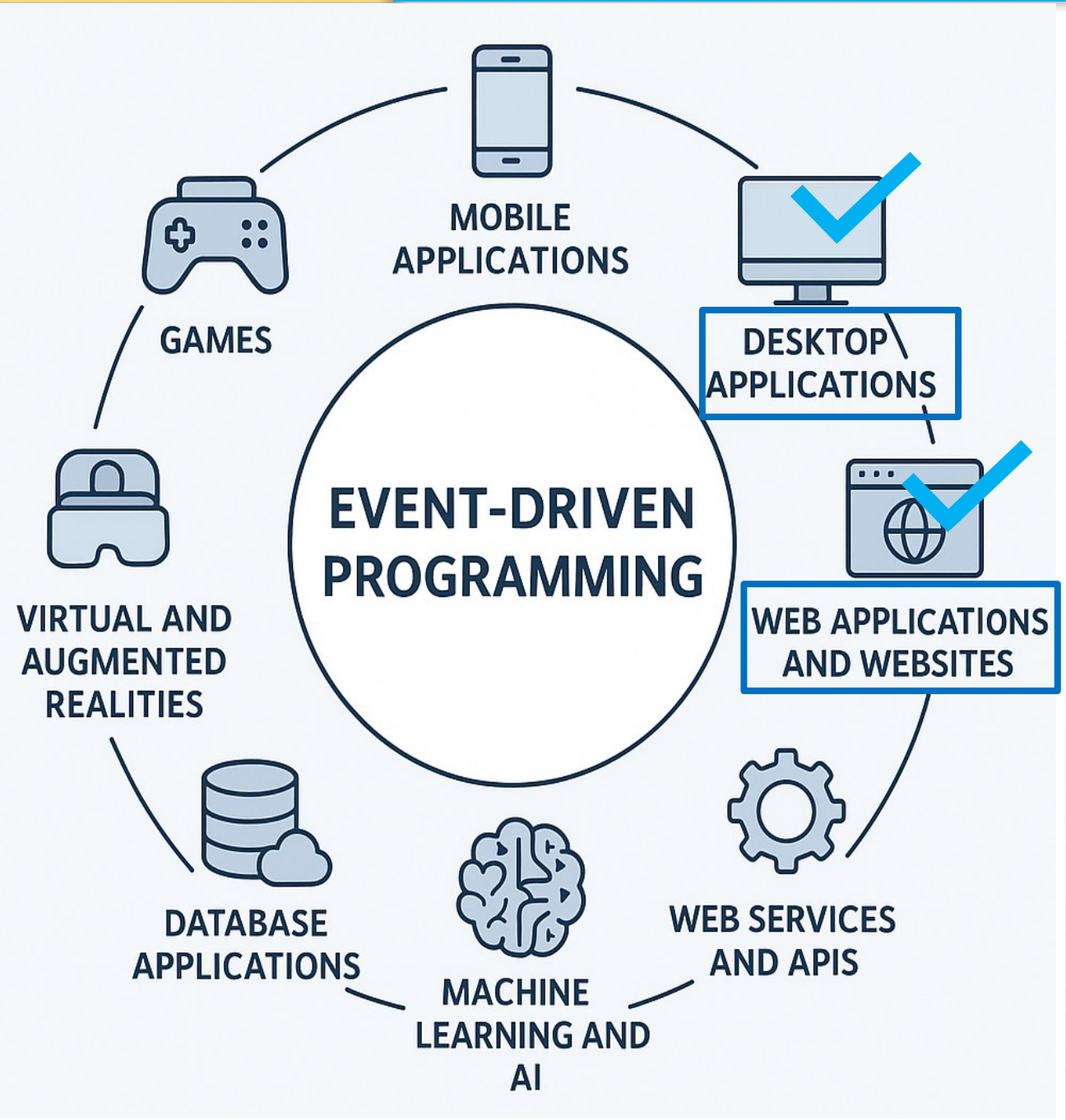

# Chapter 3: The Elements of Event-Driven Programs

By Worku Muluye

August 13, 2026

# Outline

- Windows Concepts
- GUI Design
- Windows Forms
  - Modal and Modeless
  - MDI and SDI Forms
- Controls
  - Properties
  - Methods
- Menus and Dialogs
- Event Handling (Click, Hover, …)



# Windows Concepts

- In event-driven programming, the flow of the program is determined by events such as:
  - User actions,
  - Clicking a button,
  - Hovering the mouse,
  - Typing text,
  - System messages, or sensor outputs.
- In C#, this model is widely used for building GUI applications through Windows Forms or newer frameworks like WPF and MAUI.

# Windows Concepts

- **Windows as an OS:** provides a message-driven environment where applications respond to user or system events.
- **Message Loop:** At the core of Windows applications, a loop listens for system messages such as mouse clicks, key presses, and routes them to the appropriate window or control.
- **Window Form:** A Form in C# is essentially a window, a rectangular area on the screen that can host controls and interact with the user.
- For example, when a user clicks a button on a form, Windows sends a **CLICK message**, which the .NET runtime translates into a Click event.

# GUI Design

- A Graphical User Interface (GUI) allows users to interact with software through visual indicators and widgets.
- GUI allows users to interact visually with applications rather than typing commands.
- **Good GUI Design:**
  - **Clarity and Simplicity:** The interface should be intuitive and not overwhelm the user.
  - **Consistency:** follow standard Windows conventions (similar across the application).
  - **Feedback:** the UI should respond to user actions immediately.
  - **Accessibility:** design for all users, including those with impairments.
  - **Forgiveness:** Provide ways for users to undo actions or recover from errors.

# GUI Design: GUI Design Tools in Visual Studio

- **Toolbox**: A panel in Visual Studio that contains a list of reusable controls and components that can be added to a form.
  - Provides quick access to UI elements for rapid GUI development.
  - Common Controls are Button, TextBox, Label, CheckBox / RadioButton, ComboBox / ListBox, Usage, ...
  - Visual Studio automatically generates the corresponding code in the background.
- **Designer Surface:** The visual canvas in Visual Studio where forms and their controls are arranged and customized.

# GUI Design: GUI Design Tools in Visual Studio

- It provides a What You See Is What You Get interface for designing the layout of your application without writing code manually.
  - Drag-and-drop controls from the Toolbox.
  - Resize and reposition controls visually.
  - Align, anchor, and dock controls to manage layout and responsiveness.
- It speeds up development and allows intuitive placement of controls for better user interface design.

- **Properties Window:** a panel in Visual Studio that allows developers to view and modify the properties of a selected control.
  - It customizes both the visual appearance & behavior of controls without writing code.

# Windows Forms

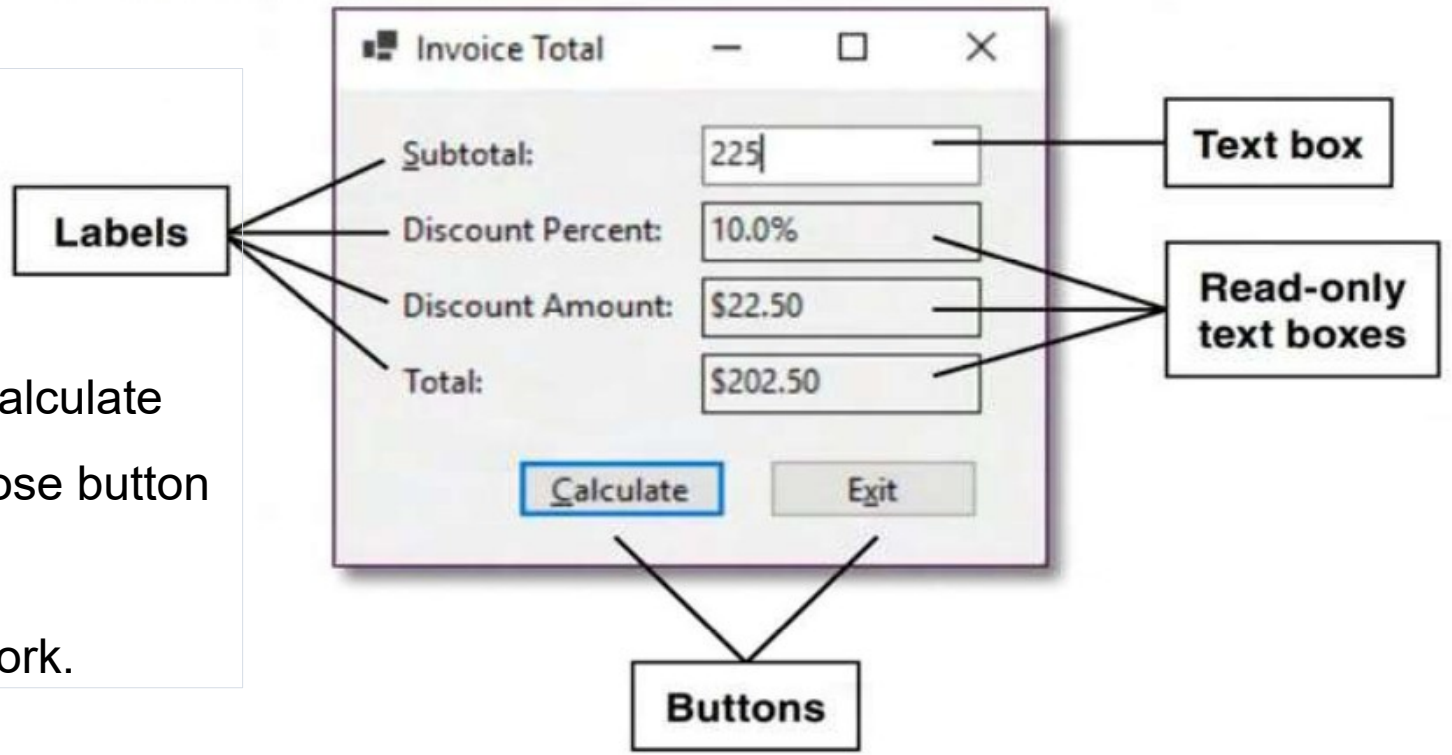
- **Windows Forms** is a .NET class library (System.Windows.Forms) for building rich Windows desktop applications, offering a drag-and-drop Visual Studio designer for efficient GUI creation.
- **Windows Forms (WinForms):** GUI framework for creating Windows desktop applications in C#.
- **Form Class:** The base container for user interface components. Every Windows Forms app starts with a form.
  - Rapid development with drag-and-drop.
  - Rich set of built-in controls.
  - Extensible with custom controls.

# Windows Forms

- For example, the invoice Total form lets users enter a subtotal, calculate discounts and total, and supports buttons or keyboard shortcuts.
- Visual Studio's Form Designer simplifies layout and auto-generates the C# code; you just add functionality.

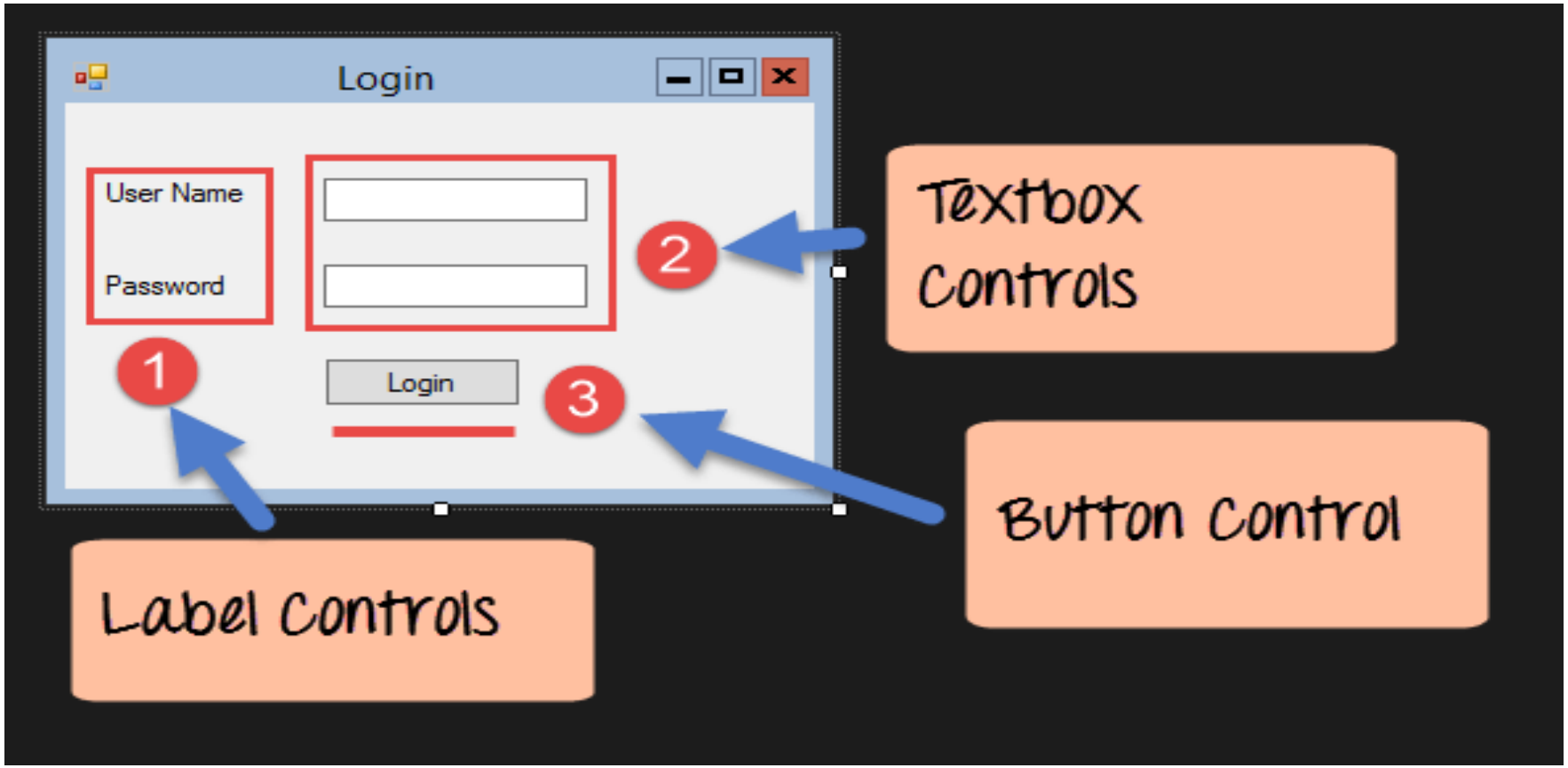
- The form uses text boxes for input/output.
- Labels to identify values.
- Buttons to perform actions.
- Users can calculate Invoice totals via the Calculate button or Enter key, and exit via the Exit/Close button or Esc key.
- Keyboard shortcuts Alt+C and Alt+X also work.

**The Invoice Total form**



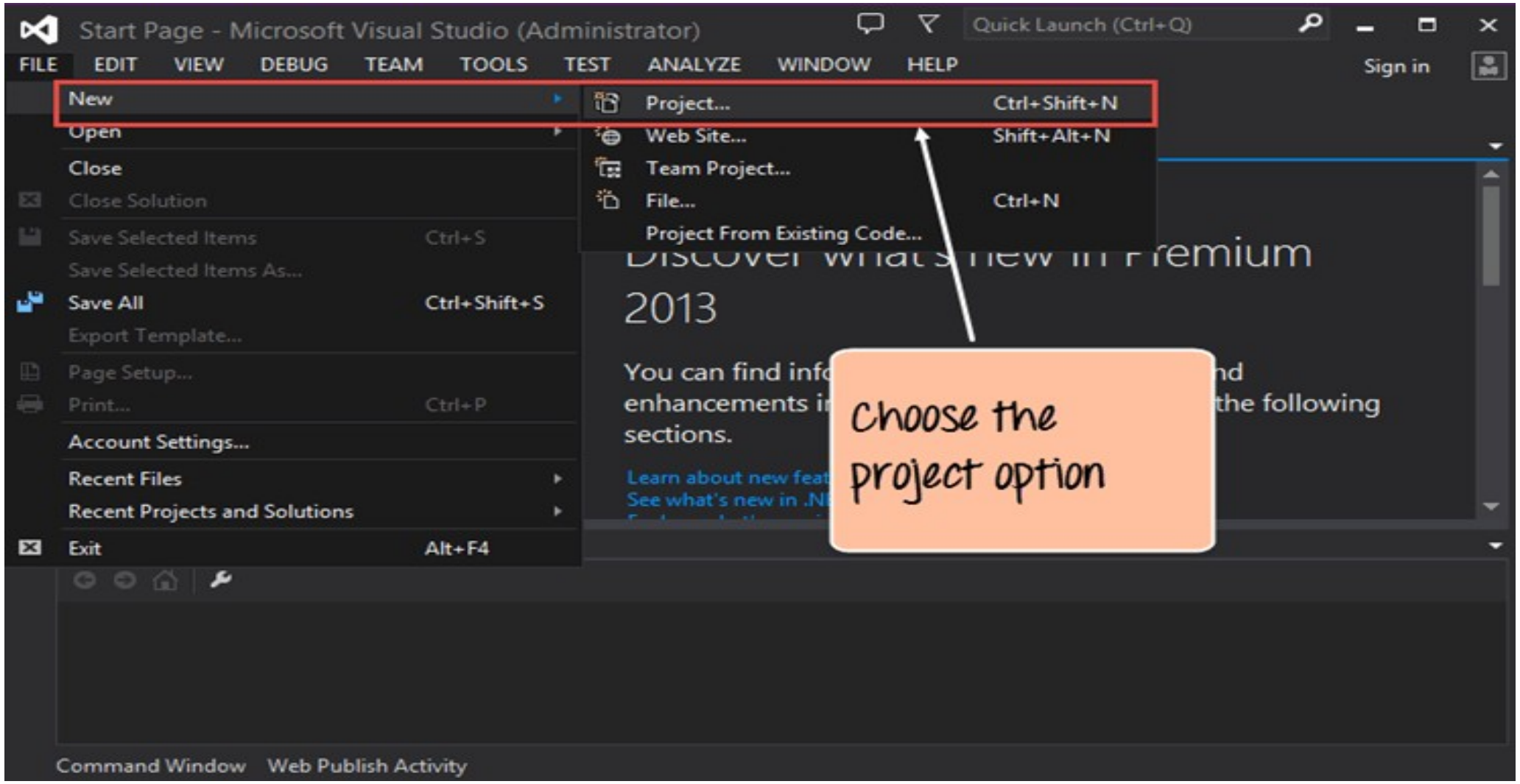
# Windows Forms

- A Windows Forms application runs on a desktop computer.
- A Windows Forms application will normally have a collection of controls such as labels, text boxes, list boxes, etc
- Example of a simple Windows form application.



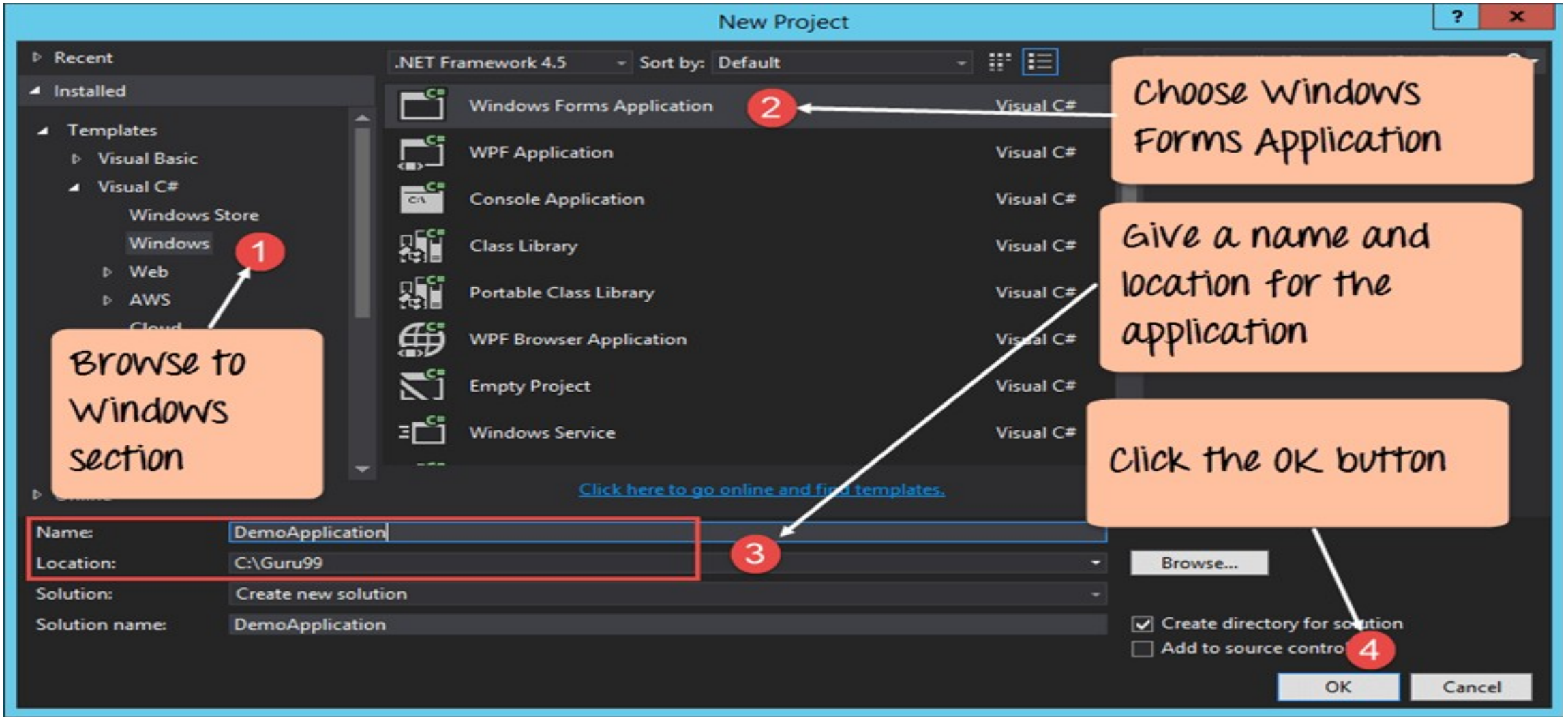
# Windows Forms

- **Step 1:** The first step involves the creation of a new project in Visual Studio. After launching Visual Studio, you need to choose the menu option New->Project.



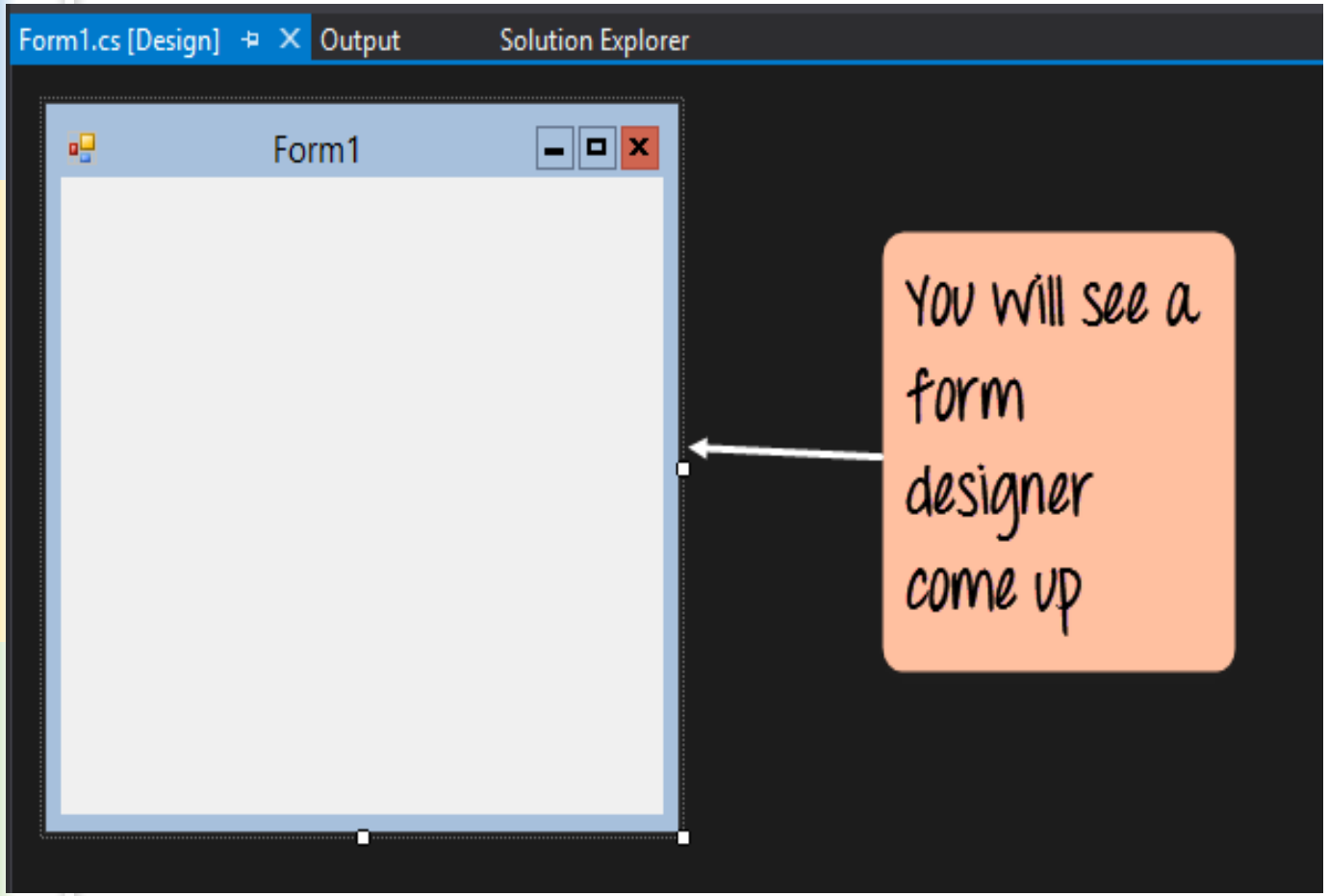
# Windows Forms

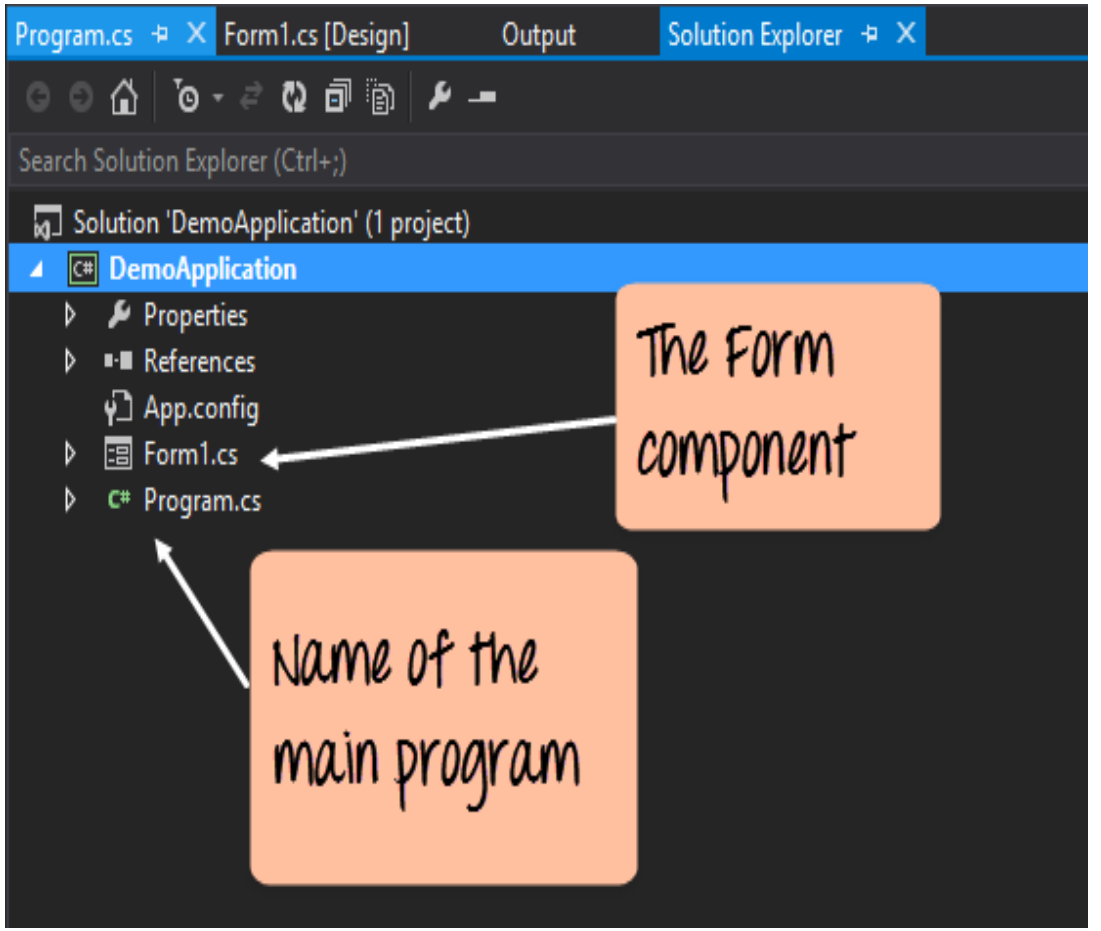
- **Step 2:** The next step is to choose the project type as a Windows Forms application. Here we also need to mention the name and location of our project.



# Windows Forms

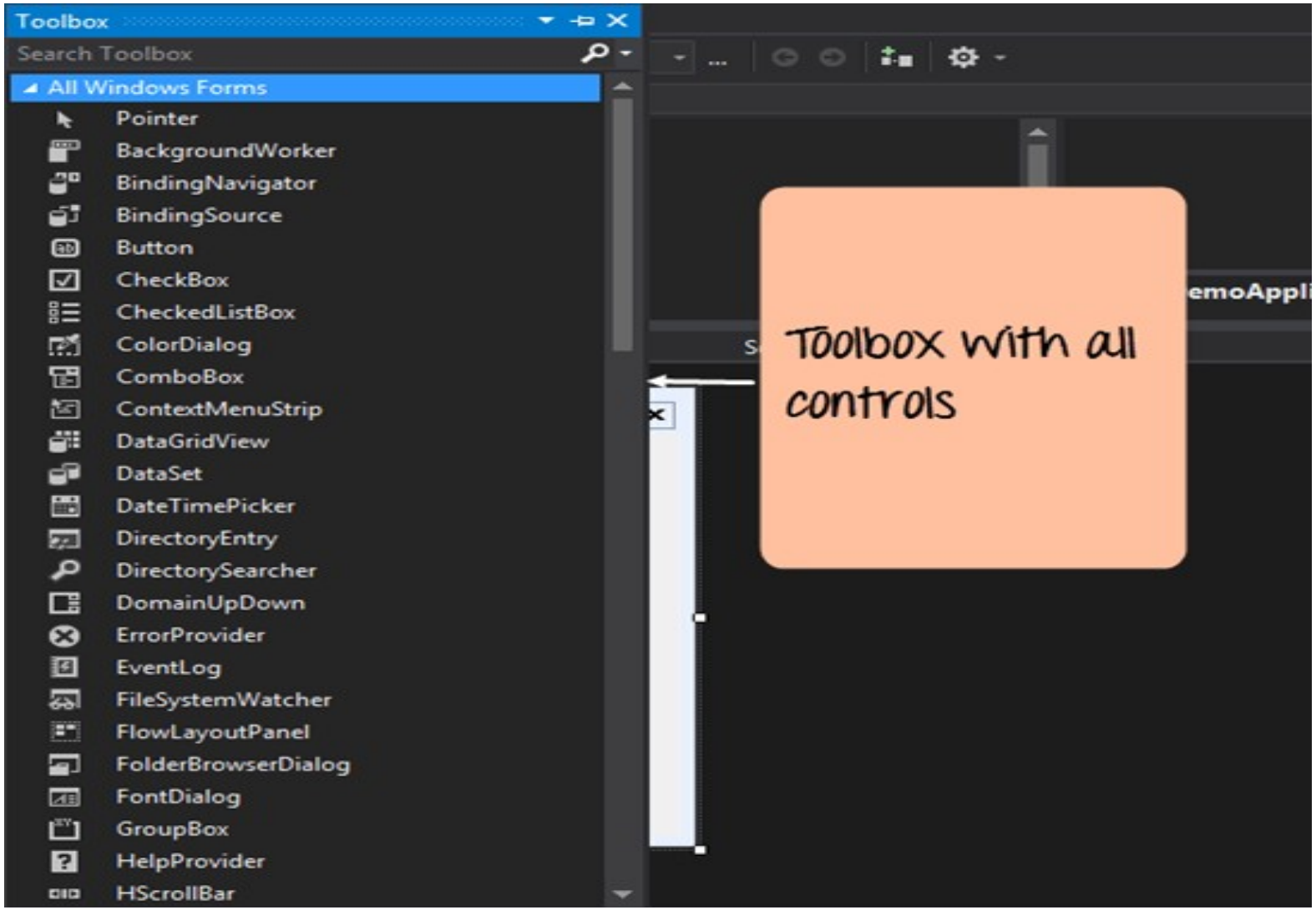
- If the above steps are followed, you will get the below output in Visual Studio.



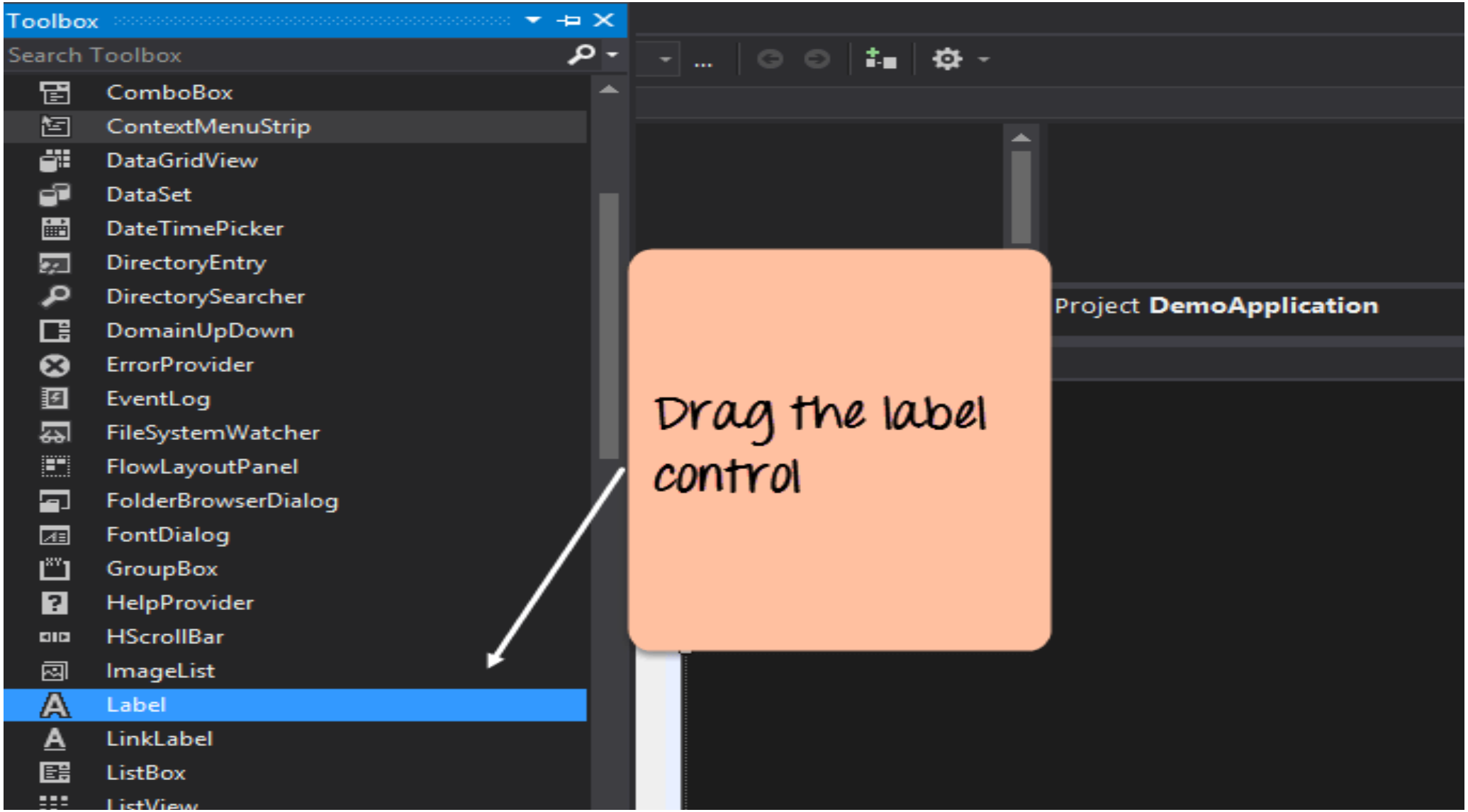
# Windows Forms

- Forms1.cs: Contains the code for the Windows Form application.
- Program.cs: The default startup file created in Visual Studio; holds the main entry code for the application.
- Toolbox: Provides controls (e.g., text box, button, label) that can be added to Windows Forms.



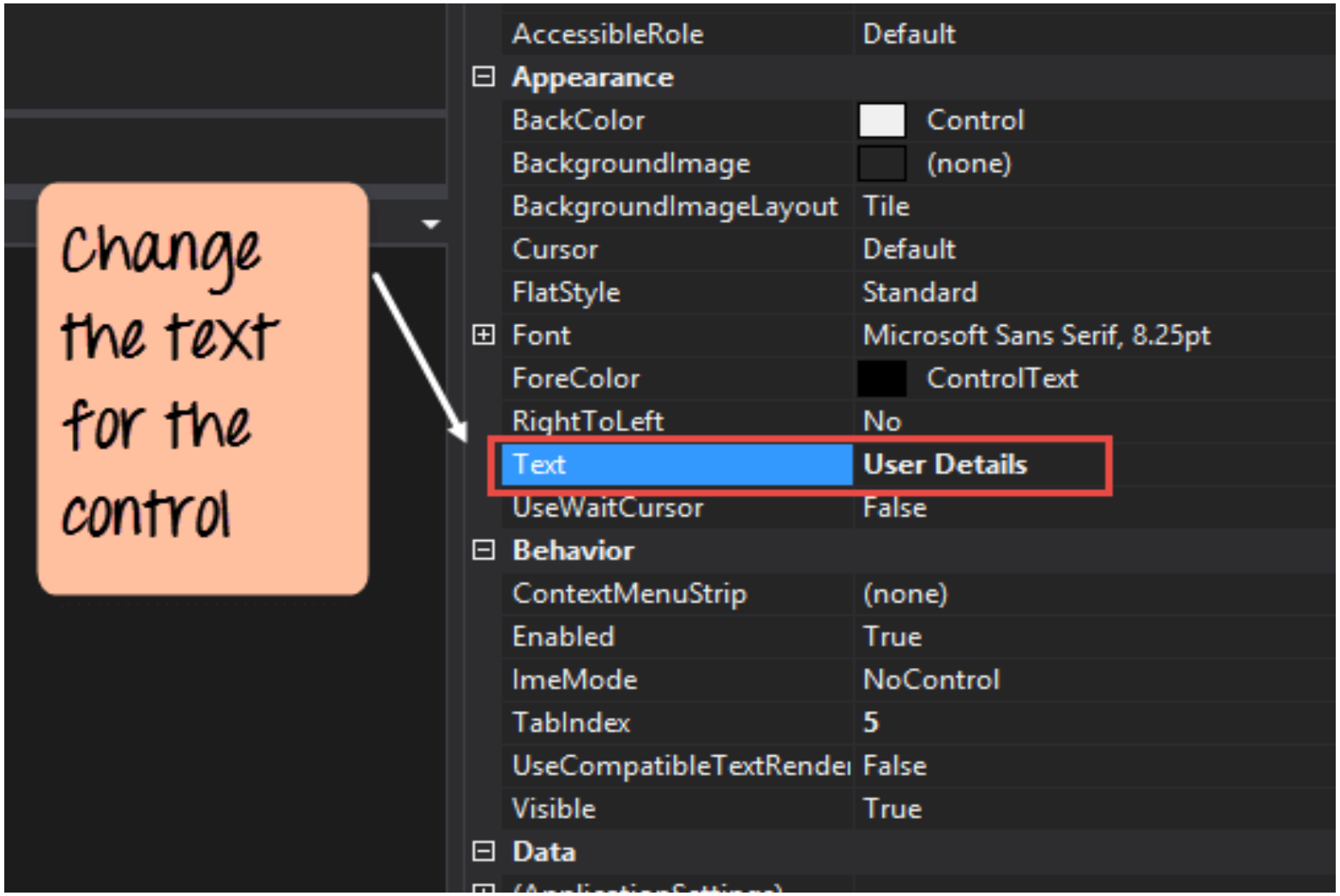
# Windows Forms

- **Step 3:** From the Toolbox, drag a Label control onto the form. It will initially display as "label".



# Windows Forms

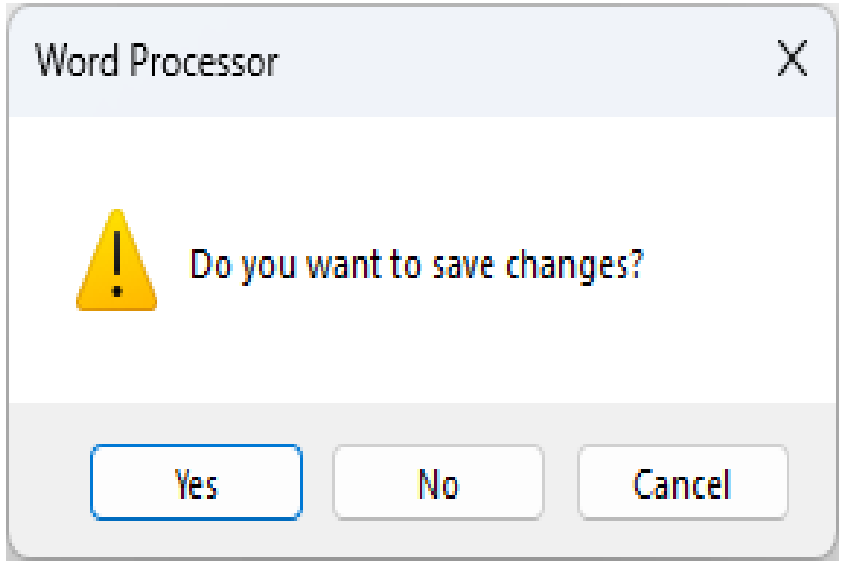
- **Step 4:** Right-click the control, select Properties, and change the Text property to 'Hello World'.



# Windows Forms: Modal Form

- The modal form requires the user to interact with it before returning to the parent form.
- It blocks interaction with the parent window that opened it until the modal form is closed.
- The user must complete the interaction with the modal dialog before returning to the main application window.
- Example: Save As, Print dialog, error messages, and any dialog that requires immediate user attention.
- Modal form displayed using the .ShowDialog() method.

```
Form2 modalForm = new Form2();
DialogResult result = modalForm.ShowDialog();
if (result == DialogResult.OK) { ... }
```

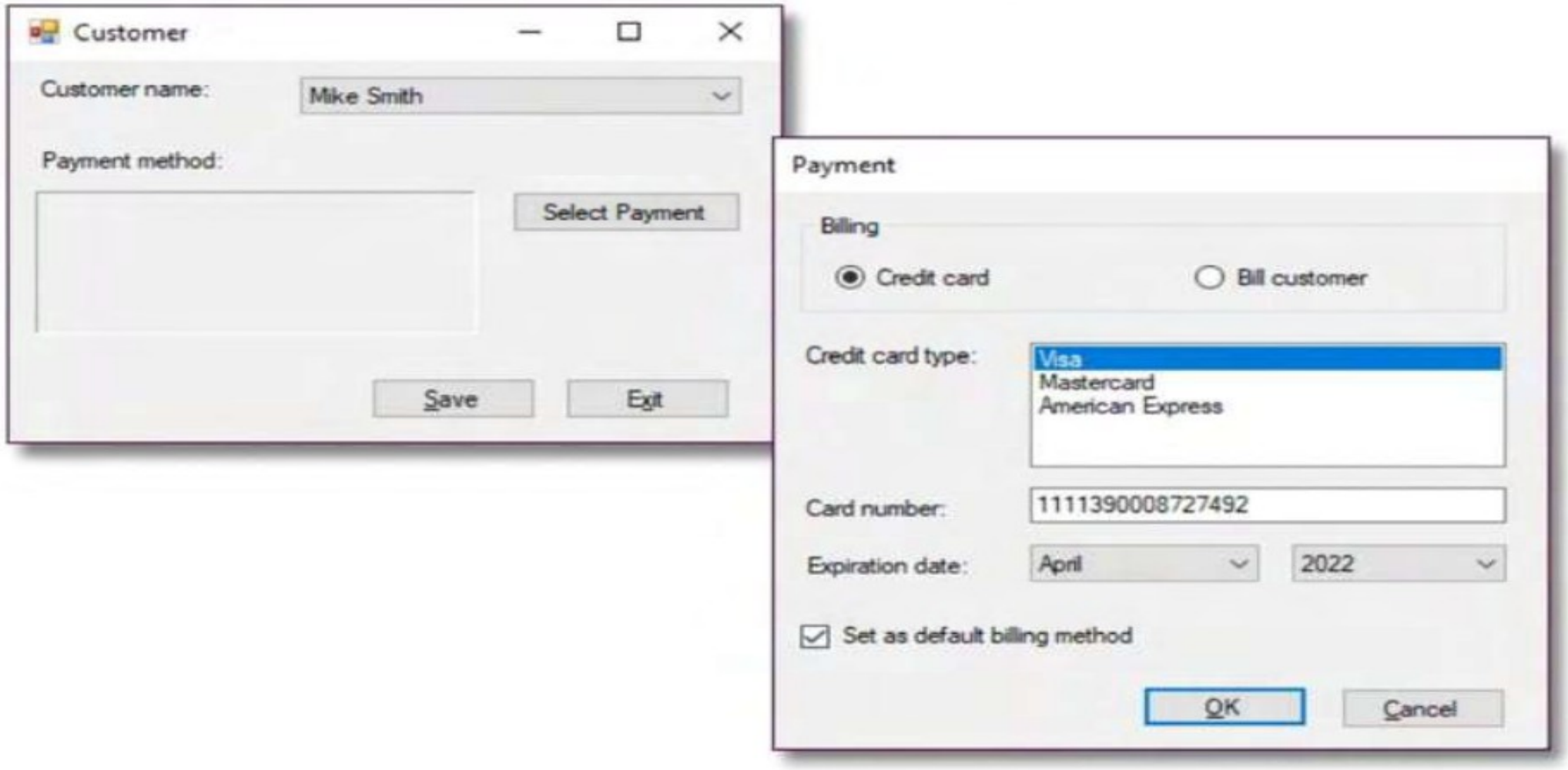


# Windows Forms: Modal Form

- How to display a form as a dialog box?
  - Dialog boxes (modal forms) are used when applications contain multiple forms and require user input before proceeding.
  - **Property Settings:** Set ControlBox = false, MaximizeBox = false, MinimizeBox = false to restrict resizing and closing options.
    - To include a Close button in the title bar: set ControlBox = true, but keep MaximizeBox = false and MinimizeBox = false.
  - **Creation and Display:** Create an instance of the form using the new keyword.
    - Display it with the ShowDialog() method.
  - **Behavior:** Application code execution pauses until the dialog box is closed.
    - Code after ShowDialog() usually processes the user's response.

# Windows Forms: Modal Form

- How to display a form as a dialog box?



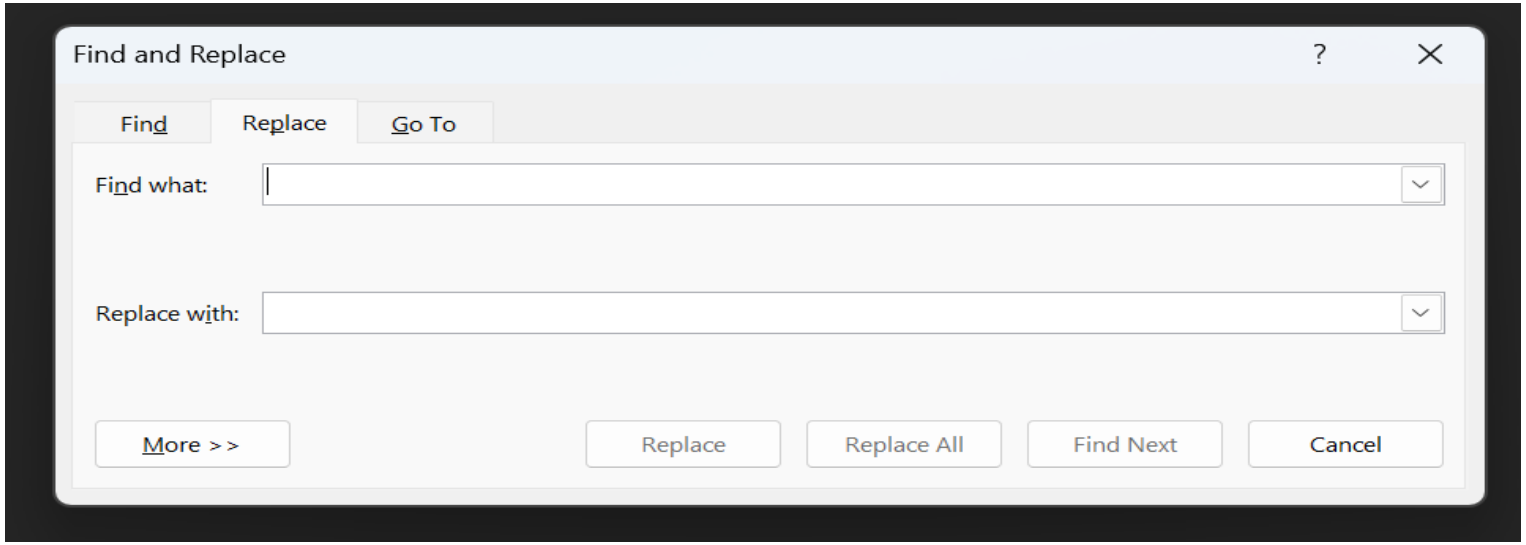
# Windows Forms: Modal Form

- Properties for creating custom dialog boxes
  - **FormBorderStyle:** Typically set to FixedDialog to prevent the user from resizing the form by dragging its border.
  - **Control1Box:** Typically set to false so the control box and the Maximize, Minimize, and Close buttons don't appear in the title bar of the form.
  - **MaximizeBox:** Typically set to false so the user can't maximize the form by double-clicking the title bar.
  - **MinimizeBox:** Can be set to false to prevent the Minimize button from being displayed if the ControlBox property is set to true.

# Windows Forms: Modeless Form

- A modeless form allows users to interact with multiple forms simultaneously without blocking the parent window. It provides a non-blocking and parallel interaction dialog box.
- The user can switch focus freely between the modeless form & other windows in the application.
- Common modeless forms are Toolboxes, palettes, floating windows.
- The Modeless form displayed using the .Show() method.
- Example: a Find and Replace dialog in a word processor, a toolbox window in Visual Studio.

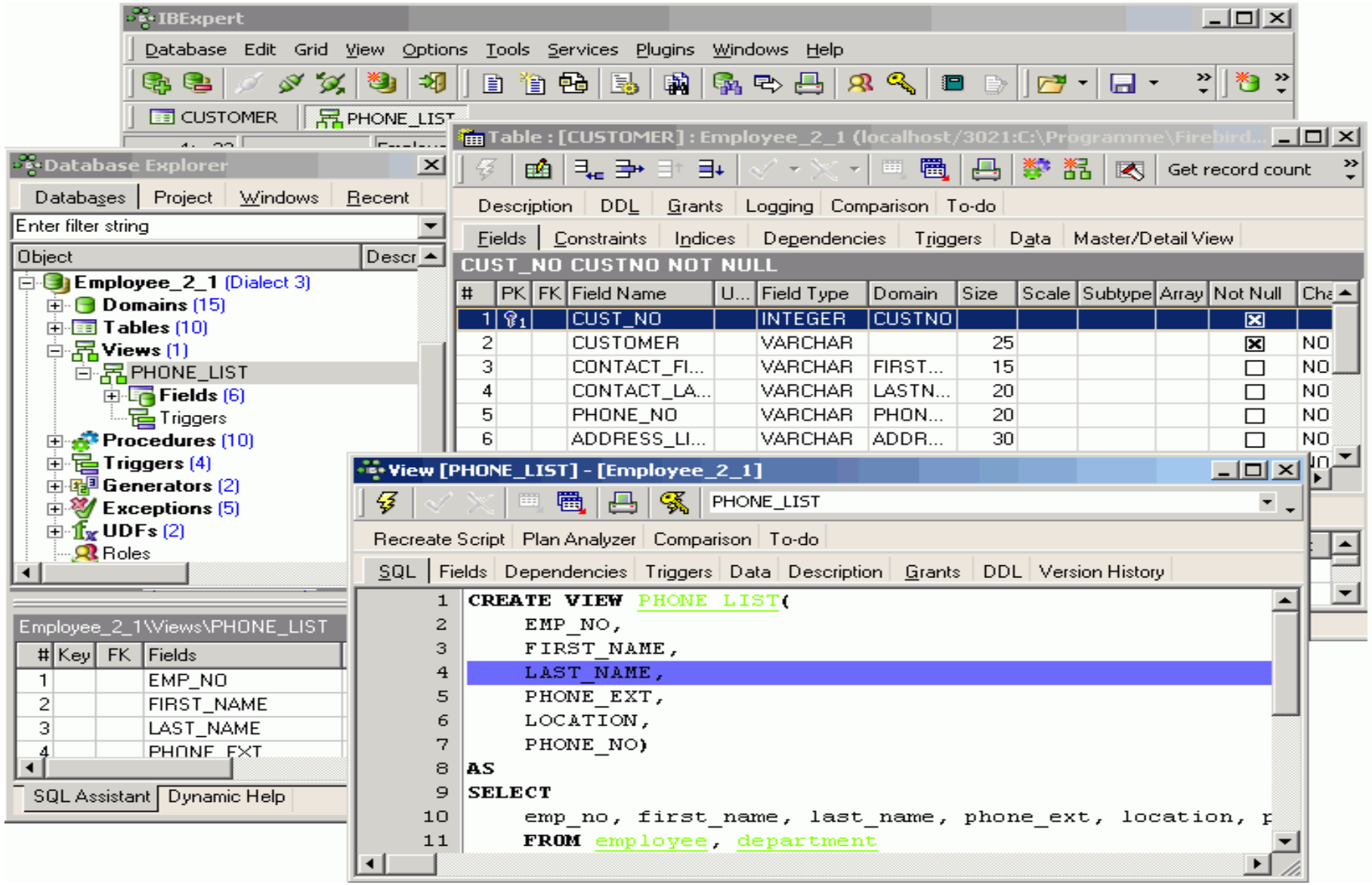
```
Form2 modelessForm = new Form2();
modelessForm.Show();
```



# Windows Forms: Single-Document Interface (SDI) Forms

- SDI is a simpler design, and each document or window is a separate instance.
- The default in Windows Forms.
- Each form is a standalone top-level window.
- Examples: recent versions of Word, Excel, and PowerPoint, Notepad, Calculator, ... If you open two Notepads or a calculator, you get two separate Notepad or calculator windows on the taskbar.
- **Advantages:**
  - Clean, simple design and less resource-intensive.
  - Suited for lightweight applications.
- **Disadvantages:**
  - Managing multiple windows can clutter the desktop.
  - Switching between documents relies on the operating system's taskbar.

# Windows Forms: SDI Forms



# Windows Forms: Multiple-Document Interface(MDI) Form

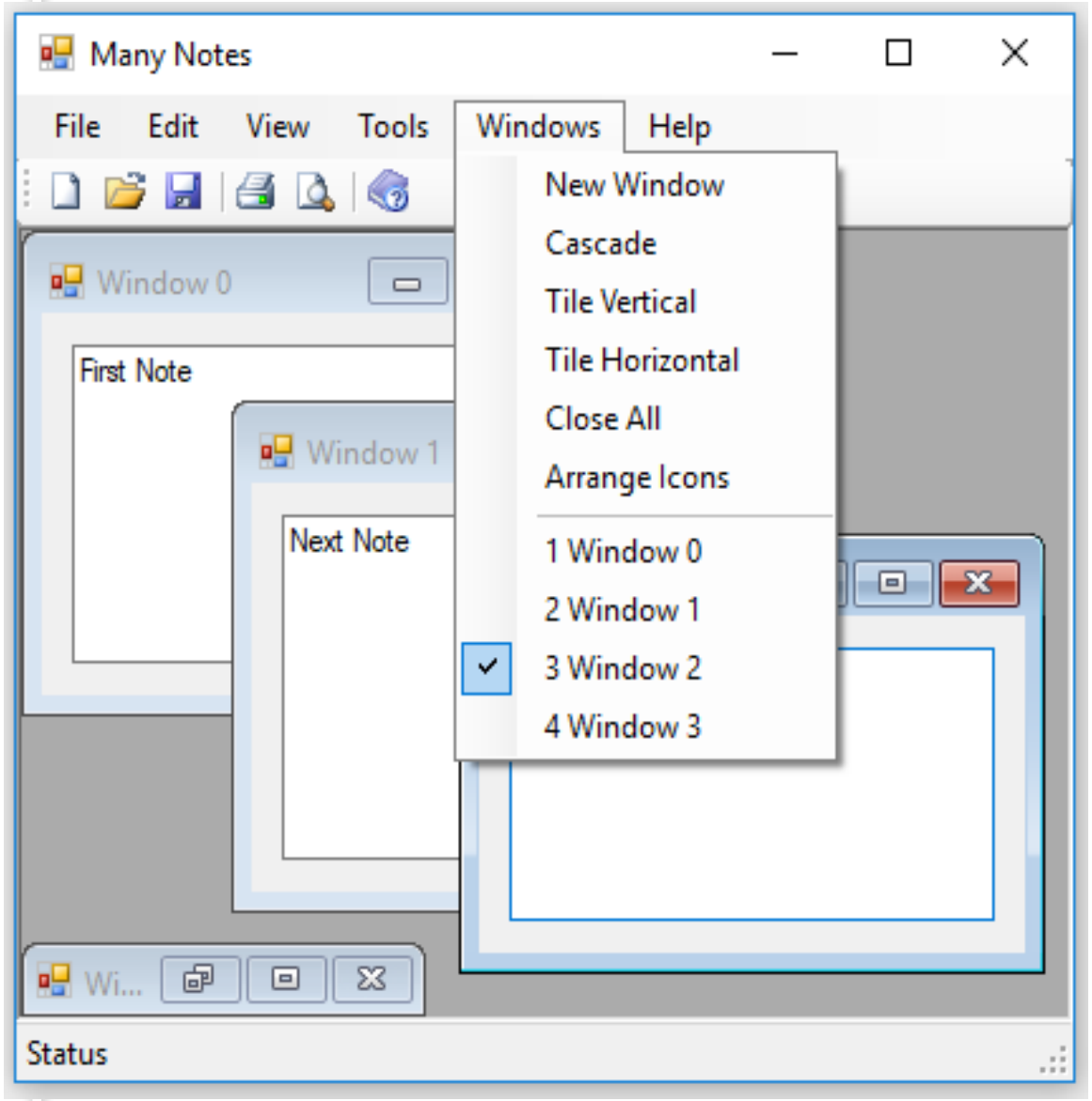
- An MDI program can display multiple child windows inside it.
- A single parent (container) window holds multiple child windows.
- The application has one main window,& documents are opened within this container.
- Example: Visual Studio Environment, older versions of MS Excel, or Photoshop.
- Set the IsMdiContainer property of the main parent form to true.
- When creating a new child form, set its MdiParent property to the main form.

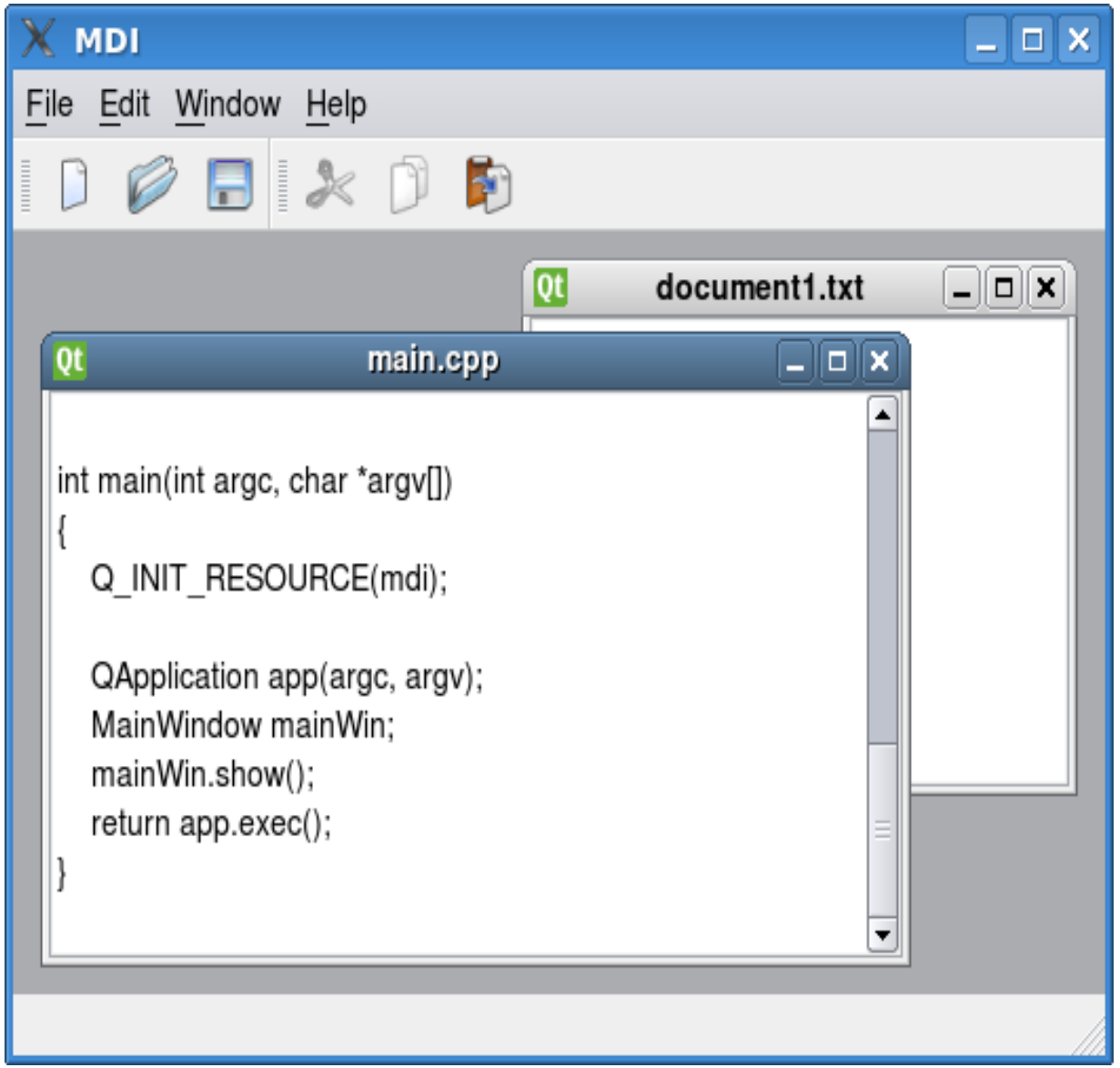
```
public partial class MainForm : Form {
    public MainForm() { InitializeComponent(); IsMdiContainer = true; }
    private void CreateChildMenuItem_Click(object sender, EventArgs e)     {
        ChildForm newChild = new ChildForm();
        newChild.MdiParent = this; // 'this' refers to the MainForm instance
        newChild.Show();  }}
```

# Windows Forms: Multiple-Document Interface(MDI) Form

- **Advantages:**
  - Easier management of multiple documents.
  - Shared menu, toolbar, and status bar across all child documents.
  - Useful for applications where users frequently work on multiple documents.
- **Disadvantages:**
  - It can become complex for very large applications.
  - Child windows may feel constrained compared to standalone windows.

# Windows Forms: MDI Form

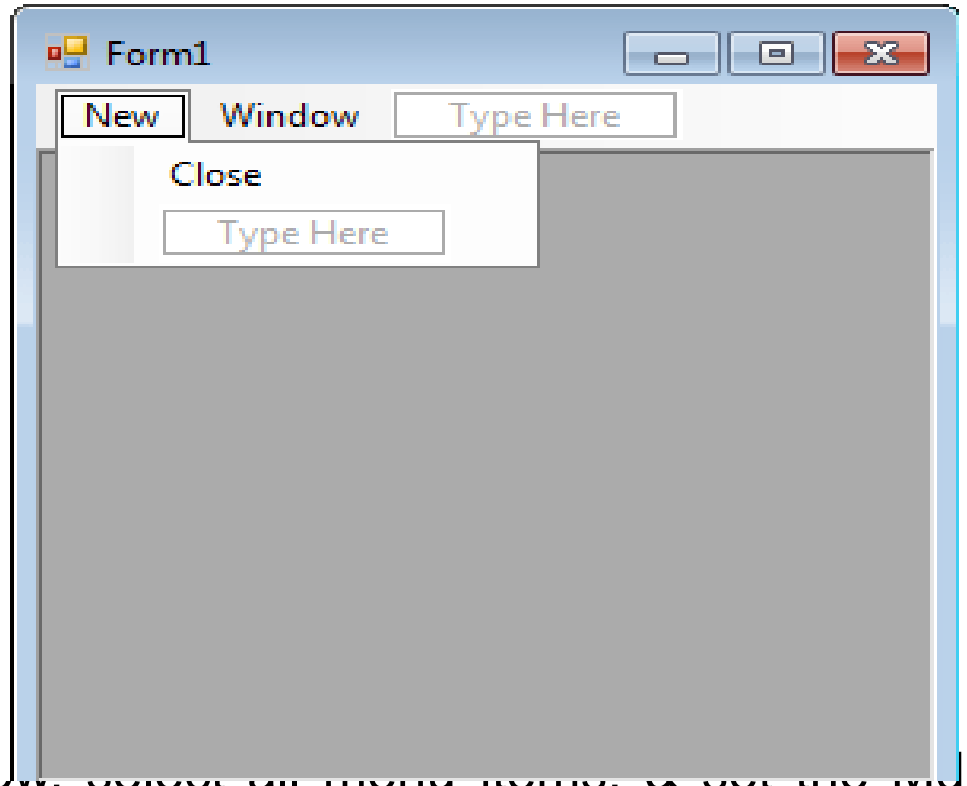


# Windows Forms: MDI Form

## Step to Create and Implement MDI Child Form

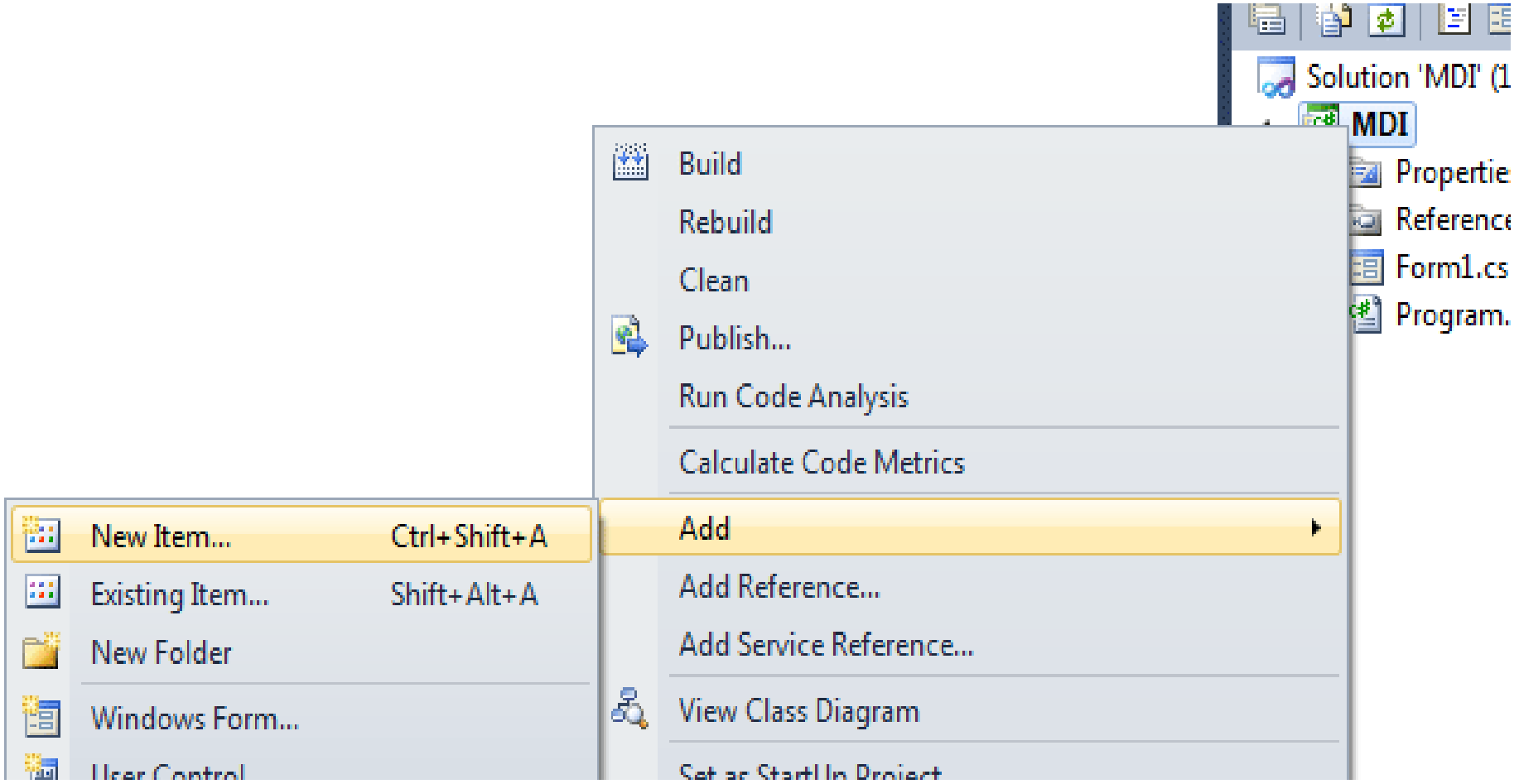
- **Step 1:** Create an MDI parent form with a MenuStrip that contains the values New and Windows, and Close in the sub menu of New.




- **Step 2:** Now go to the Properties Wind[illegible] select all menu items, & set the MdiList property to true.

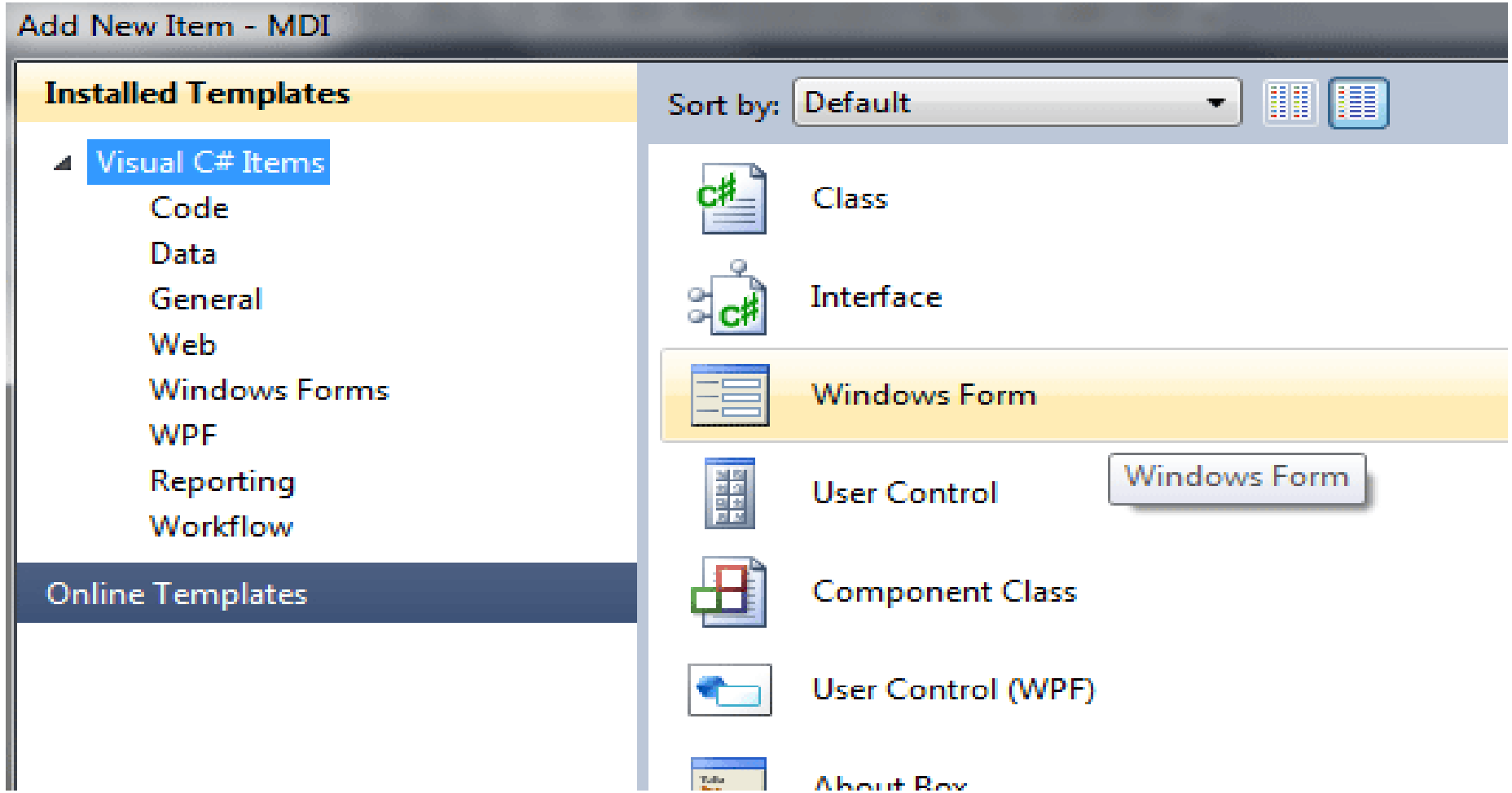
# Windows Forms: MDI Form

**Step 3:** Now add MDI child forms by going to Solution Explorer, right-click the project, select Add New Item from the Add Menu.



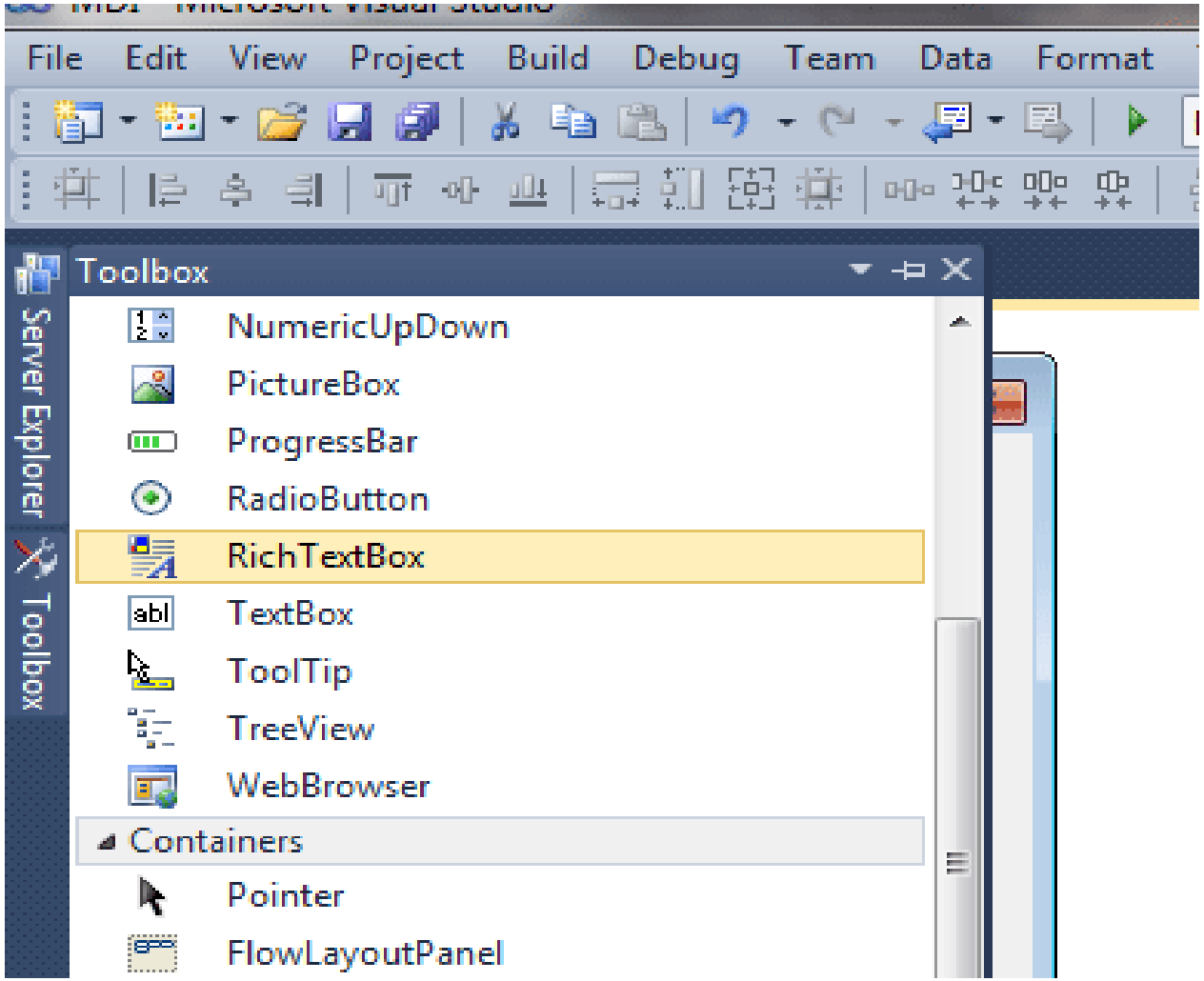
# Windows Forms: MDI Form

**Step 4:** A Dialog box appears, now select Windows Form, give the name as Form2, and click on the open button. A New Form Name as Form2 added.



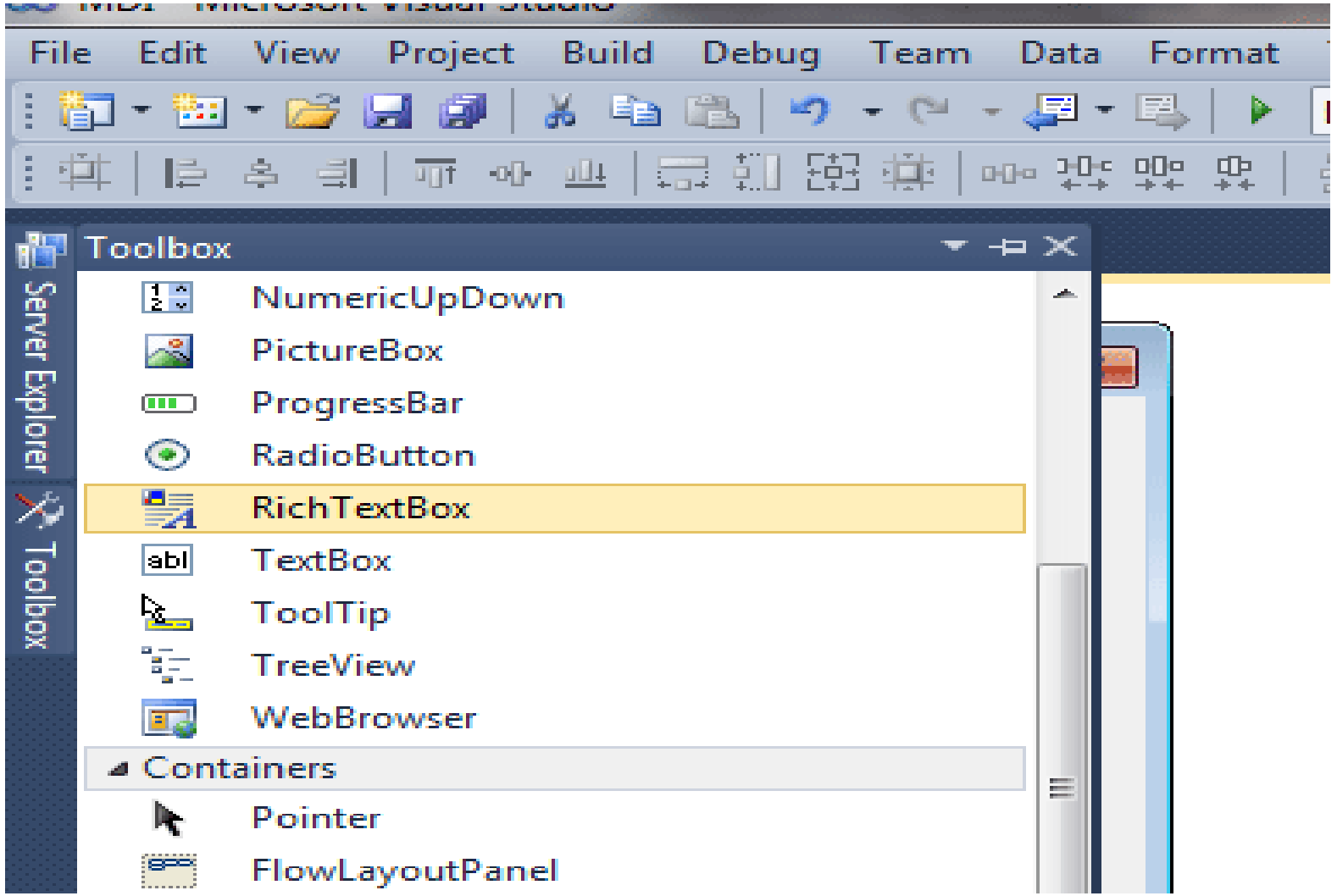
# Windows Forms: MDI Form

**Step 5:** Add a RichTextBox Control to the Form from the Toolbox.



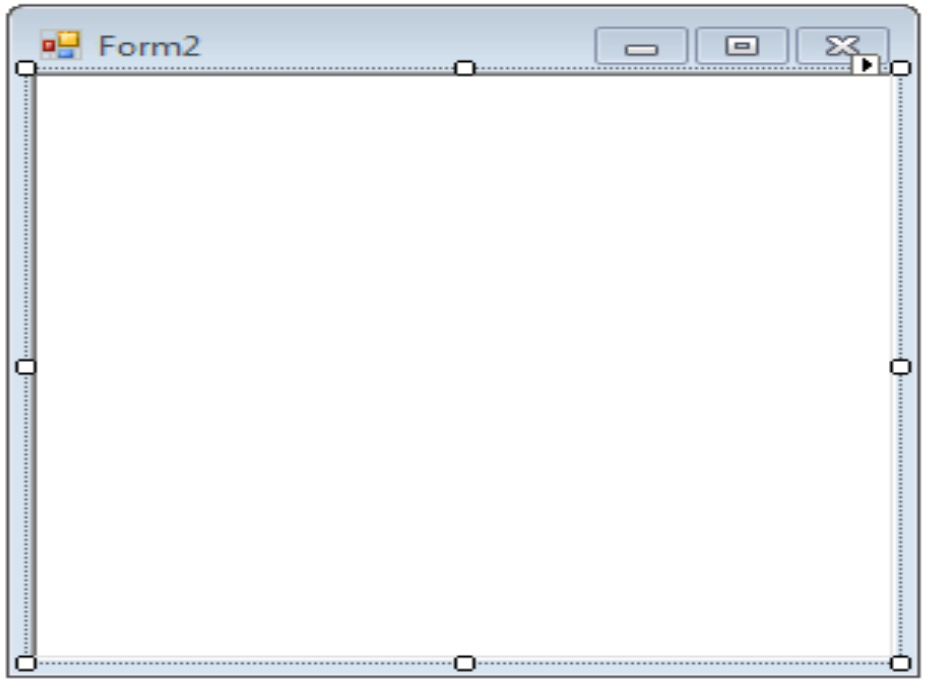
# Windows Forms: MDI Form

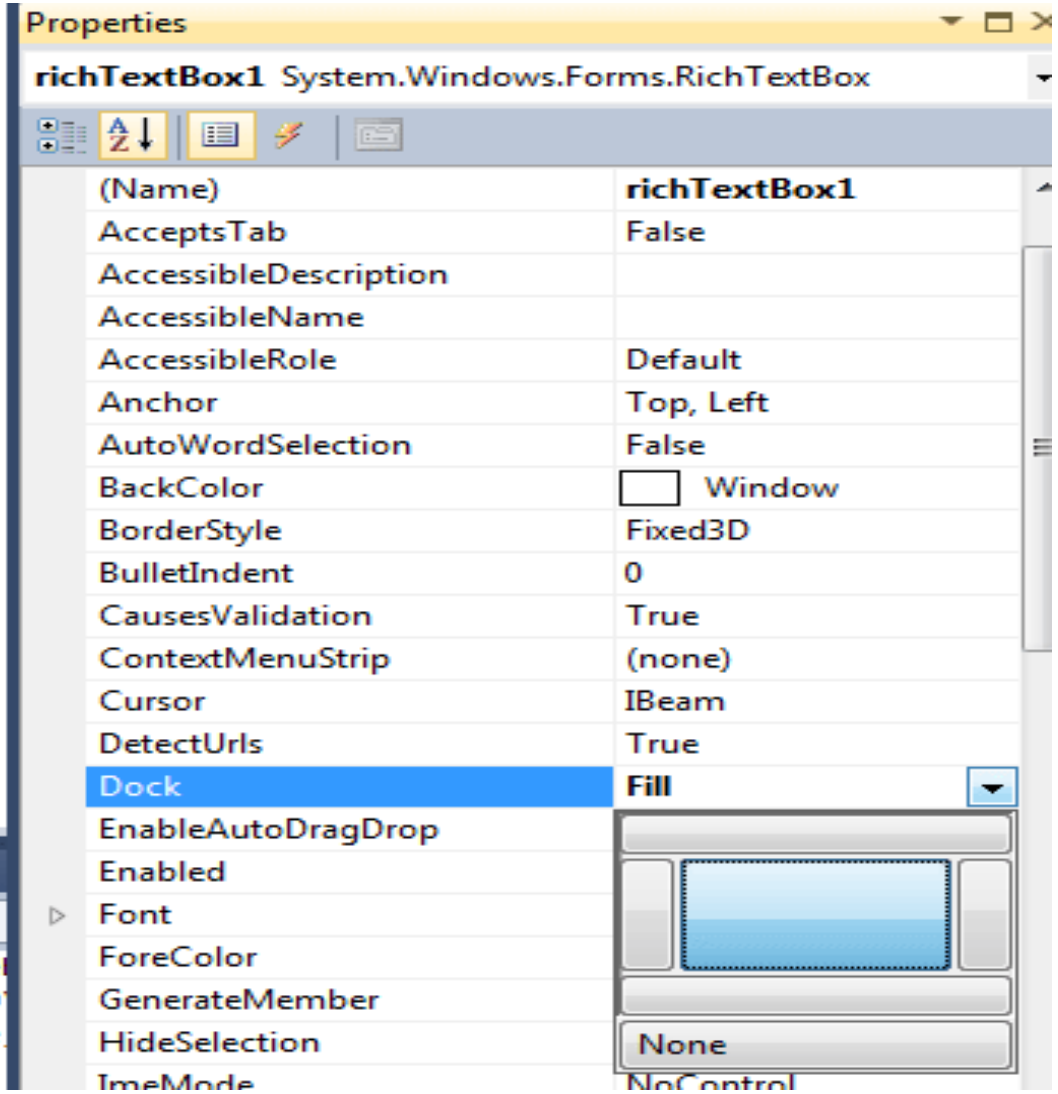
**Step 5:** Add a RichTextBox Control to the Form from the Toolbox.

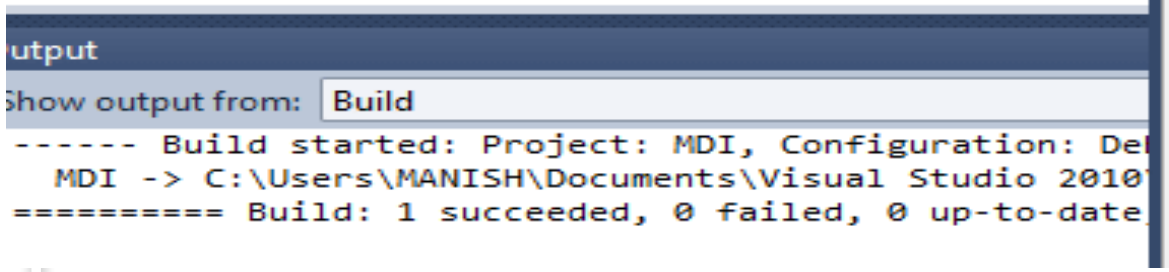


# Windows Forms: MDI Form

**Step 6:** Now, go to the Property Window & Set Properties for the RichTextbox. Set the Anchor property to Top, Left, and the Dock property to Fill.

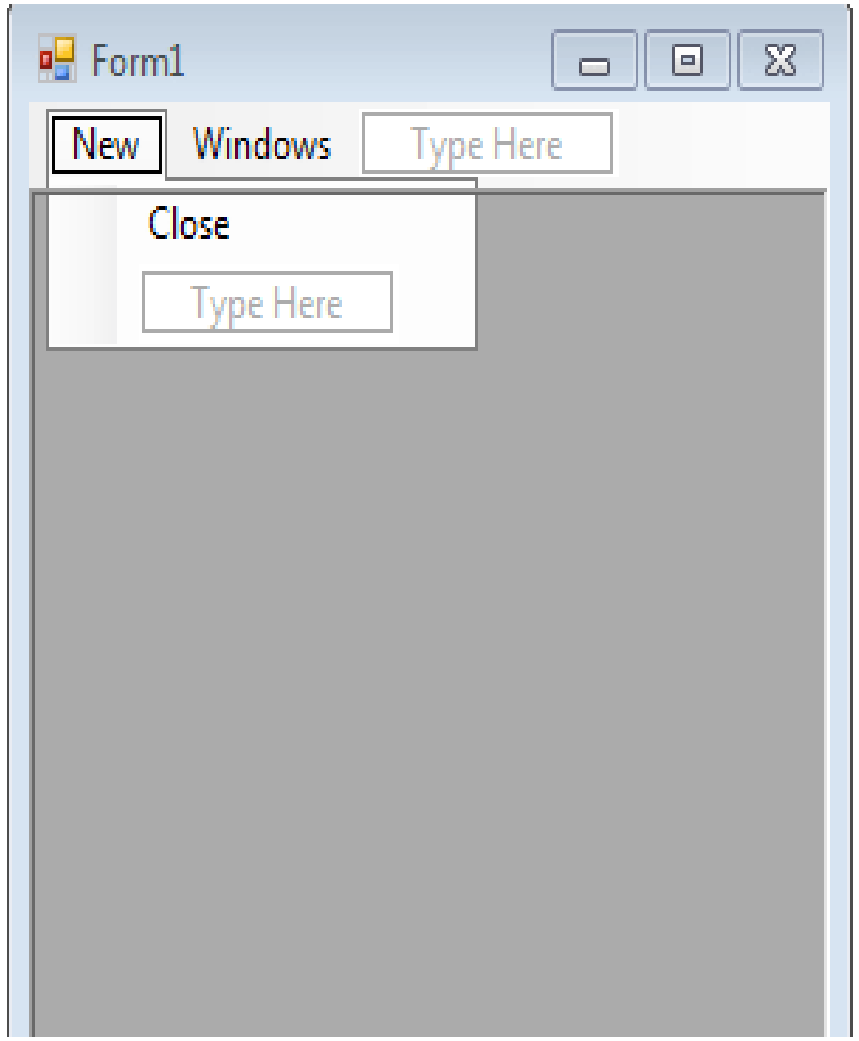


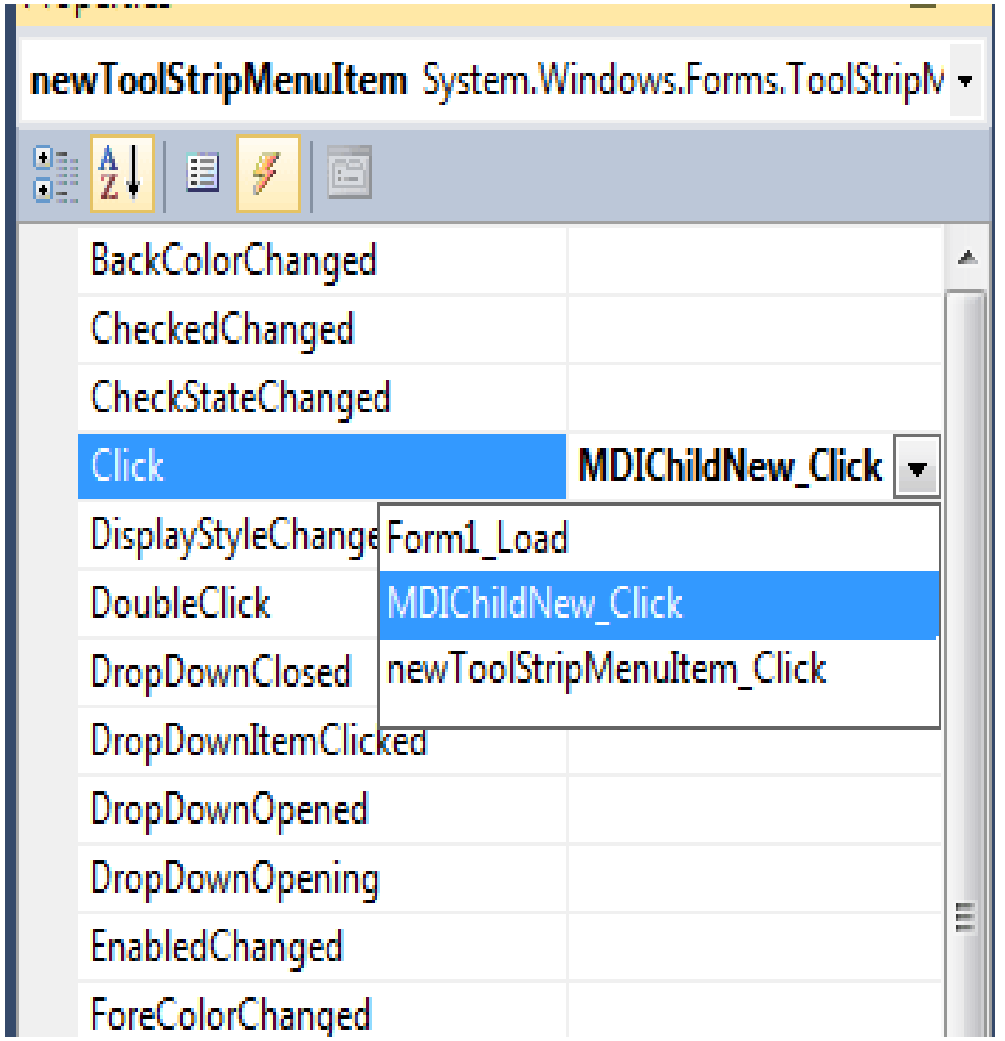



# Windows Forms: MDI Form

**Step 7:** Click on New Menu Item and generate a Click Event Handler on it.



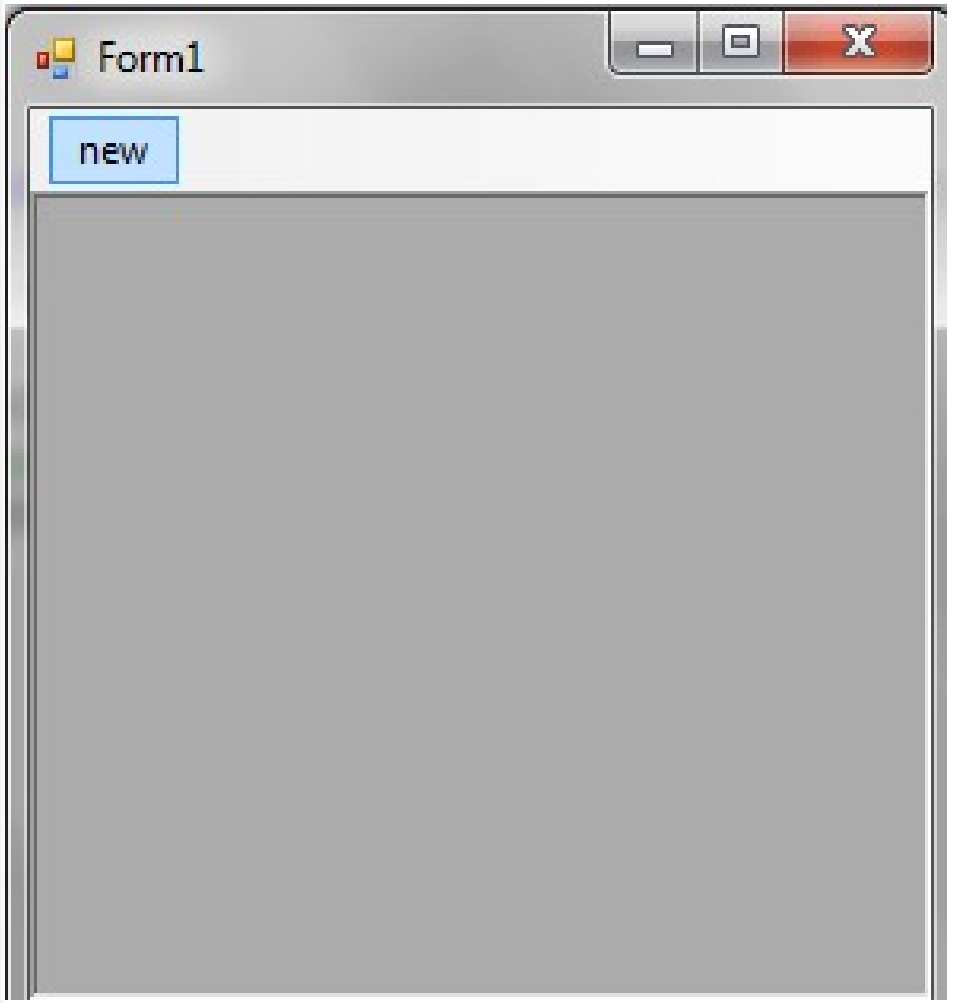
# Windows Forms: MDI Form

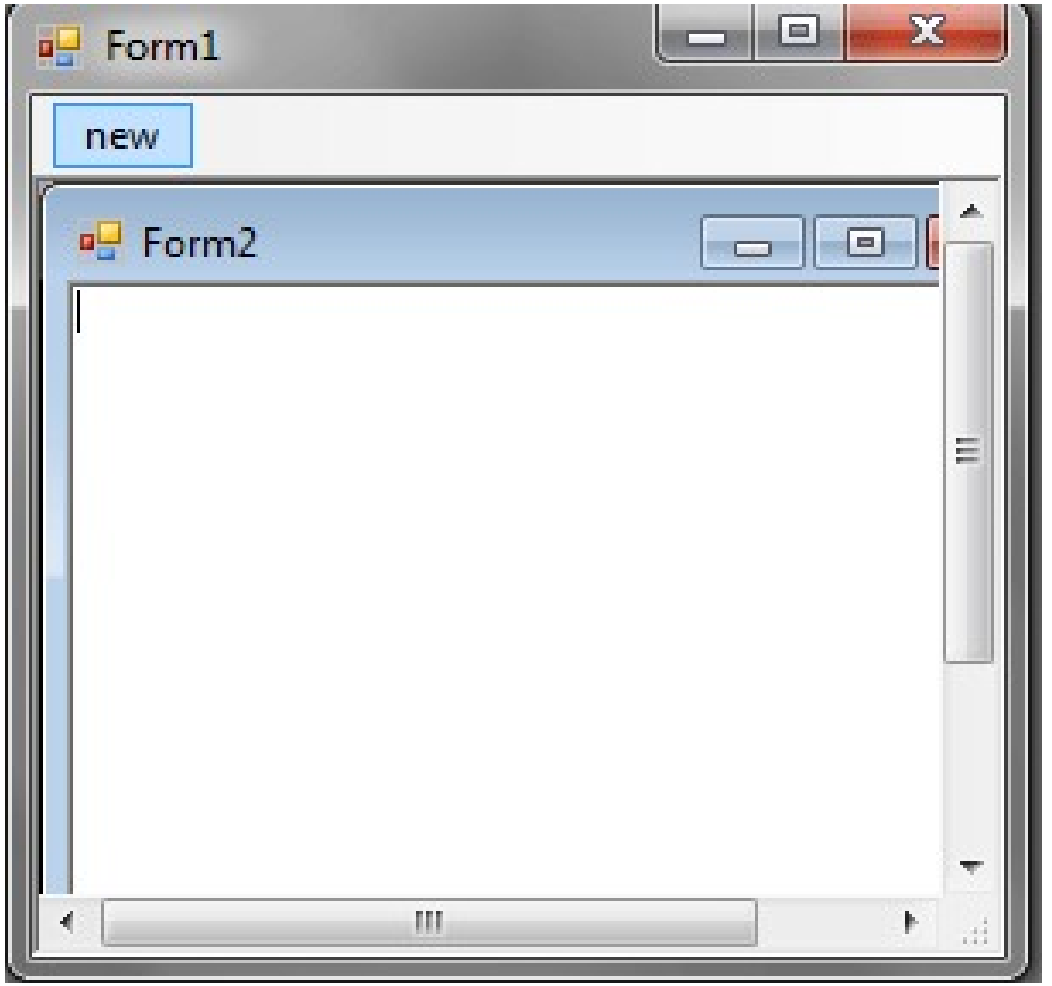
**Step 8:** Now double-click on New Menu Item and write down the following Code.

```
private void newToolStripMenuItem_Click(object sender, EventArgs e)
{
    Form2 newMDIChild = new Form2();
    newMDIChild.MdiParent = this; //set the Parent Form of the Child window
    newMDIChild.Show(); //display the new form
}
protected void MDIChildNew_Click(object sender, EventArgs e)
{
    Form2 newMDIChild = new Form2();
    newMDIChild.MdiParent = this; //set the Parent Form of the Child window
   newMDIChild.Show();  //display the new form
}
```

# Windows Forms: MDI Form

**Step 9:** Now Run the application, And from output window you can create a new MDI child form by clicking on New Menu Item.



# Windows Forms: Multi-Frame Interface (MFI)

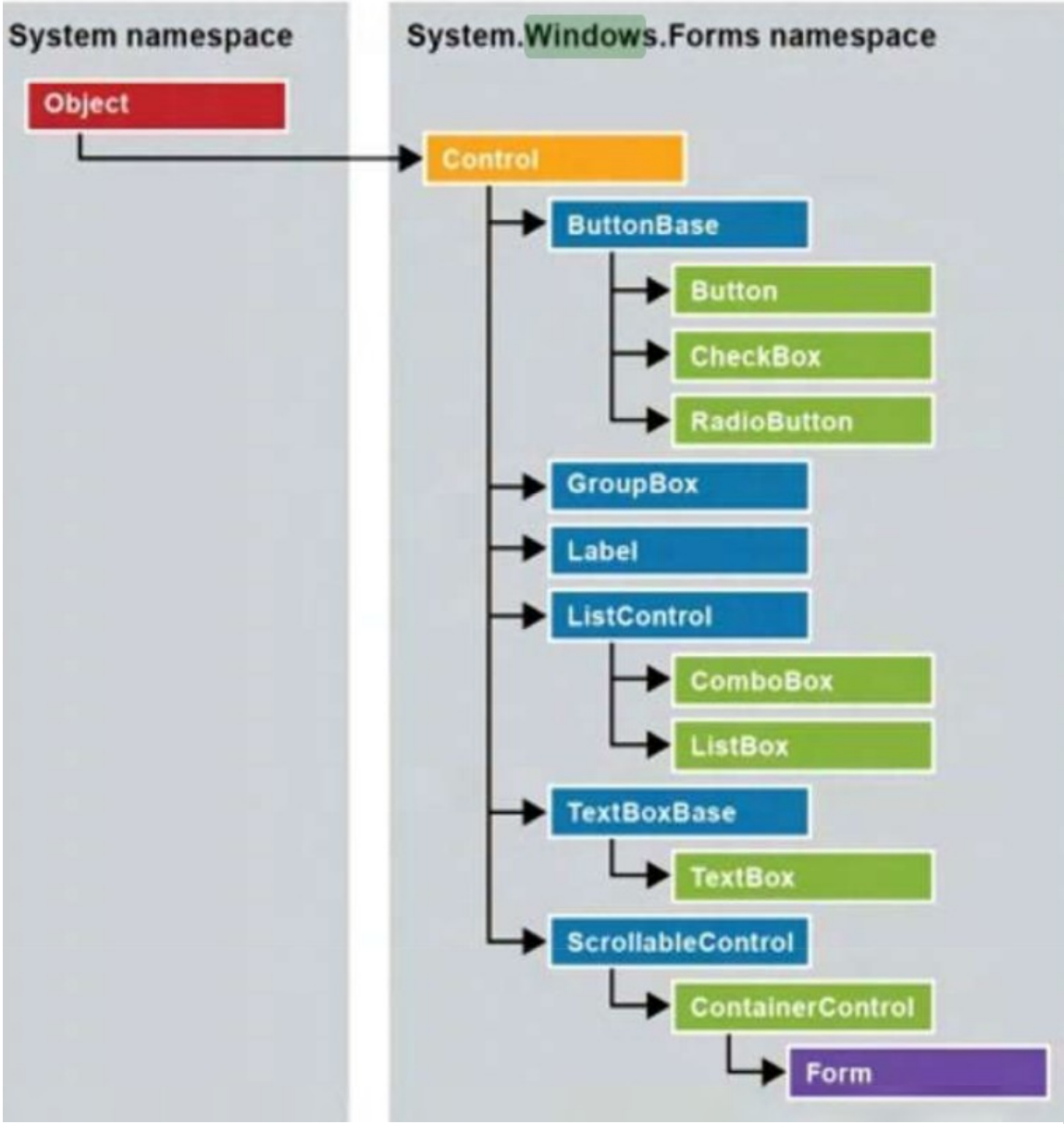
- MFI is a user interface design where multiple independent frames/windows are part of a single application but can run side by side, often with cooperative communication.
- MFI is similar to SDI in that each document has its own window.
- Unlike pure SDI, frames may be linked together and can share data.
- Each frame can have its own menu and controls.
- **Advantages:**
  - Allows users to arrange windows freely (e.g., tile, cascade).
  - More flexible than MDI because windows aren't restricted to a parent form.
  - Provides a balance between SDI and MDI.
- **Disadvantages:**
  - Higher complexity in managing multiple frames.
  - Risk of UI clutter if too many frames are opened.
  - For example, modern web browsers (Google Chrome, Mozilla Firefox, each tab is like a frame but can run separately), Eclipse IDE.

# Controls

- Controls are GUI elements placed on a form that allow users to interact with an application.
- They act as the building blocks of a Windows Forms application.
- Control has properties(behavior settings), methods (actions it can perform), and events(responses to user/system actions).
- **Types of Controls:**

**A. Basic Controls**

- **Label**: Displays static text or instructions (read-only).
- **TextBox**: Accepts user input in the form of text.
- **Button**: Executes an action when clicked.
- **PictureBox**: Displays images such as .jpg, .png, or .bmp.



# Controls

**B. Container Controls:** These hold and organize other controls.

- **GroupBox**: Groups related controls with a border and title.
- **Panel**: A rectangular container that can host multiple controls, often used for layout.
- **TabControl:** Provides a tabbed interface for organizing content in separate pages.

**C. Advanced Controls:** Used for displaying and managing complex data or hierarchical information.

- **DataGridView:** Displays and edits tabular data.
- **TreeView:** Displays a hierarchical data example, folder structure.
- **ListView:** Displays a list of items in different views (details, icons, tiles).
  - For example, File Explorer, TreeView for folders, and ListView for file details.

# Controls

**TextBox Control:**

- A text box object is used to display text on a form or to get user input while a C# program is running.
- In a text box, a user can type data or paste it into the control from the clipboard.
- For displaying a text in a TextBox control

```
textBox1.Text = “DBU-IS-2018";
```

- Collect the input value from a TextBox control into a variable

```
string var;
var = textBox1.Text;
```

- C# TextBox Properties

```
textBox1.Width = 250;
textBox1.Height = 50
```

# Controls

**TextBox Control:**

- **Background Color and Foreground Color:**

```
textBox1.BackColor = Color.Blue;
textBox1.ForeColor = Color.White;
```

- **Textbox BorderStyle:** You can set 3 different types of border style for the textbox, they are None, FixedSingle, and fixed3d.

```
textBox1.BorderStyle = BorderStyle.Fixed3D;
```

- **Textbox Maximum Length:** Sets the maximum number of characters or words the user can input into the text box control.

```
textBox1.MaxLength = 40;
```

- **Textbox ReadOnly:** When a program wants to prevent a user from changing the text that appears in a text box, the program can set the control's Read-only property to True.

```
textBox1.ReadOnly = true;
```

# Controls

**TextBox Control:**

- **Multiline TextBox:** Use the Multiline and ScrollBars properties to enable multiple lines of text to be displayed or entered.

```
textBox1.Multiline = true;
```

- **Textbox password character:** TextBox controls can also be used to accept passwords and other sensitive information. To mask characters entered in a single-line version of the control, use the PasswordChar property.

```
textBox1.PasswordChar = '*';
```

- How to retrieve values from a textbox?
  - int i; i = int.Parse (textBox1.Text);
  - float i; i = float.Parse (textBox1.Text);
  - double i; i = float.Parse (textBox1.Text);

# Controls

## ComboBox Control

- A ComboBox combines a TextBox and a ListBox.
- Users can select from the list or enter a custom value.
- Items are added using Items.Add():

```
comboBox1.Items.Add("Sunday");
```

- Set default selection:

```
comboBox1.SelectedIndex = comboBox1.FindStringExact("Sunday");
```

- Access selected value on button click:

```
string var = comboBox1.Text;
MessageBox.Show(var);
```

# Controls

## CheckBox Control

- Allows users to select one or more options.
- It supports binary choices.
  - Yes/No, True/False.
- It can display text, images, or both.
- Example: checkBox1.Text = "Apple";
- Optional Three-State mode:
  - Checked,
  - Unchecked,
  - Indeterminate.
- Enable it with:
  - checkBox1.ThreeState = true;

```
private void button4_Click(object sender, EventArgs e){
    string msg = "";
    if (checkBox1.Checked)
        msg += "checkBox1 ";
    if (checkBox2.Checked)
        msg += "checkBox2 ";
    if (checkBox3.Checked)
        msg += "checkBox3 ";
    if (msg.Length > 0)
        MessageBox.Show(msg + "selected");
    else
        MessageBox.Show("No checkbox selected");
    //enable three-state mode for checkBox1
    checkBox1.ThreeState = true;
}
```

# Controls

## RadioButton Control

- It allows the user to select only one option from a group.
- Selecting one radio button automatically unchecks the others in the same group.
- It can display text, an image, or both. Use the Checked property to get or set its state:

```
if (radioButton1.Checked) {
    //block of code
}
```

```
private void button1_Click(object sender, EventArgs e){
    if (radioButton1.Checked) {
        MessageBox.Show("You selected Red!");
        return;
    }
    else if (radioButton2.Checked) {
        MessageBox.Show("You selected Blue!");
        return;
    }
    else {
        MessageBox.Show("You selected Green!");
        return;
    } }
```

# Controls

## ListBox Control

- ListBox displays a scrollable list of items from which users can select.
- Users click an item to select it (highlighted when chosen).
- Supports both single and multiple selection modes:
  - Single allows one item at a time.
  - Multiple lets users select more than one using Ctrl or Shift keys.
- Commonly used when you want to show several options at once, rather than hiding them behind a dropdown.

# Controls

**ListView:**

- The ListView control is an ItemsControl that is derived from ListBox.

**DateTimePicker:**

- The DateTimePicker control allows you to display and collect dates and times from the user with a specified format.

# Controls: Properties

- Properties are attributes of a control that define its appearance, behavior, and state at runtime or design time.
- They act like variables with special accessors that allow you to configure a control without writing explicit methods.
- Properties can be set visually through the Properties Window in Visual Studio, or programmatically in C# code.
  - Sets or retrieves the text displayed on a control(text).

    ```
    btnSubmit.Text = "Submit";
    ```
  - Changes the background color of the control(BackColor).

    ```
    btnSubmit.BackColor = Color.Blue;
    ```
  - Changes the text (foreground) color(ForeColor).

    ```
    lblTitle.ForeColor = Color.Red;
    ```
  - Determines whether the control is active and usable (true/false)(Enabled).

    ```
    txtInput.Enabled = false;
    ```

# Controls: Properties

- Controls whether the control is shown or hidden (true/false) (Visible).

```
panel1.Visible = true;
```

- Font defines the text's style, size, and typeface.

```
lblTitle.Font = new Font("Arial", 14, FontStyle.Bold);
```

- Size Sets the width and height of a control.

```
btnSubmit.Size = new Size(100, 40);
```

- Location defines the X, Y position of the control on the form.

```
btnSubmit.Location = new Point(50, 120);
```

# Controls: Methods

- Methods are actions (functions) that a control can perform.
- Unlike properties (which define a control's state or attributes), methods trigger behavior or cause something to happen.
- Methods can be called programmatically during runtime to manipulate controls and forms.

# Controls: Methods

- Show(): displays a form or control.

```
form2.Show();
```

- Hide(): makes a form or control invisible but does not destroy it.

```
form2.Hide();
```

- Focus(): sets the input focus to a specific control.

```
txtName.Focus();
```

- Clear(): clears the text or contents of an input control (e.g., TextBox).

```
txtInput.Clear();
```

- Refresh(): redraws the control to reflect updates immediately.

```
panel1.Refresh();
```

- Dispose(): releases all resources used by a control.

```
form2.Dispose();
```

- Close(): Closes a form.

```
this.Close();
```

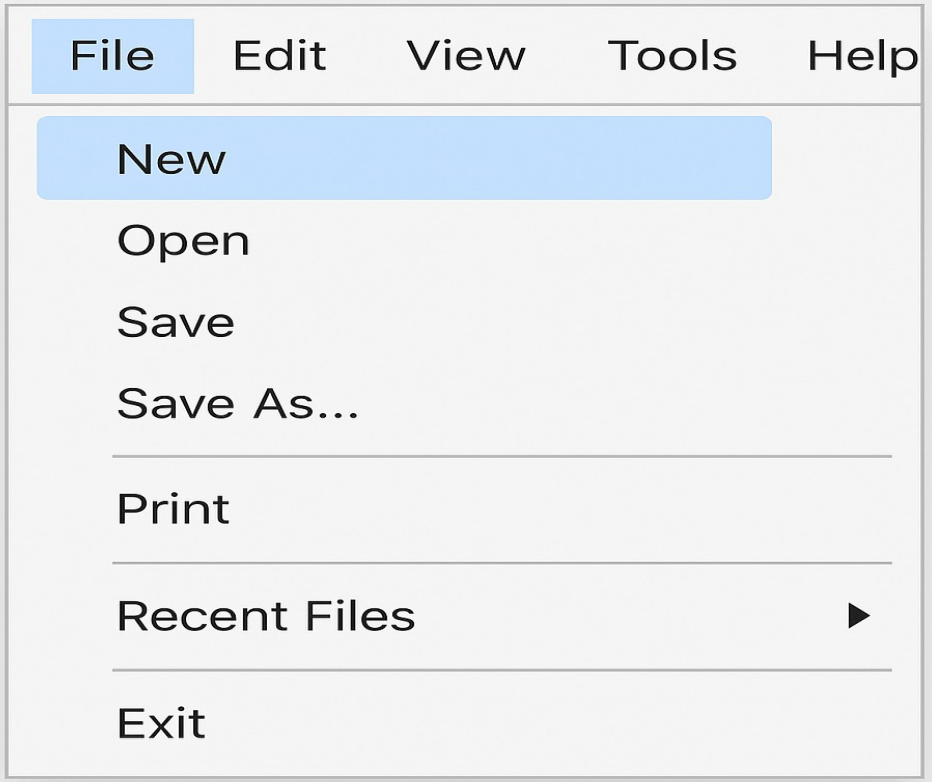
# Menus

- Menus are **structured sets of commands** in applications.
- They provide users with an **organized way to navigate** and perform tasks.
- Often displayed at the **top of the window (Menu bar)** or as a **context menu (right-click)**.
- **Types of Menus:**
  - **MenuStrip:** Standard menu bar control, such as File, Open, Save, Exit.
  - Supports drop-down menus and sub-menus.
  - **ContextMenuStrip:** A right-click menu that provides shortcuts to commands. Context-sensitive (options vary depending on the control).

# Menus

- Example,

```
//adding MenuStrip with File Menu
MenuStrip menuStrip = new MenuStrip();
ToolStripMenuItem fileMenu = new ToolStripMenuItem("File");
fileMenu.DropDownItems.Add("Open");
fileMenu.DropDownItems.Add("Save");
fileMenu.DropDownItems.Add("Exit");
menuStrip.Items.Add(fileMenu);
this.MainMenuStrip = menuStrip;
this.Controls.Add(menuStrip);
```



# Dialogs

- Dialogs are pre-built modal forms that help perform common tasks such as file selection, color picking, and font settings.
- They are typically modal, meaning the user must finish the dialog before returning to the main form.
- **Common Dialogs in Windows Forms:**
- **OpenFileDialog:** Browse and select a file to open.

  openFileDialog: ShowDialog();
- **SaveFileDialog:** Save a file with a given name.

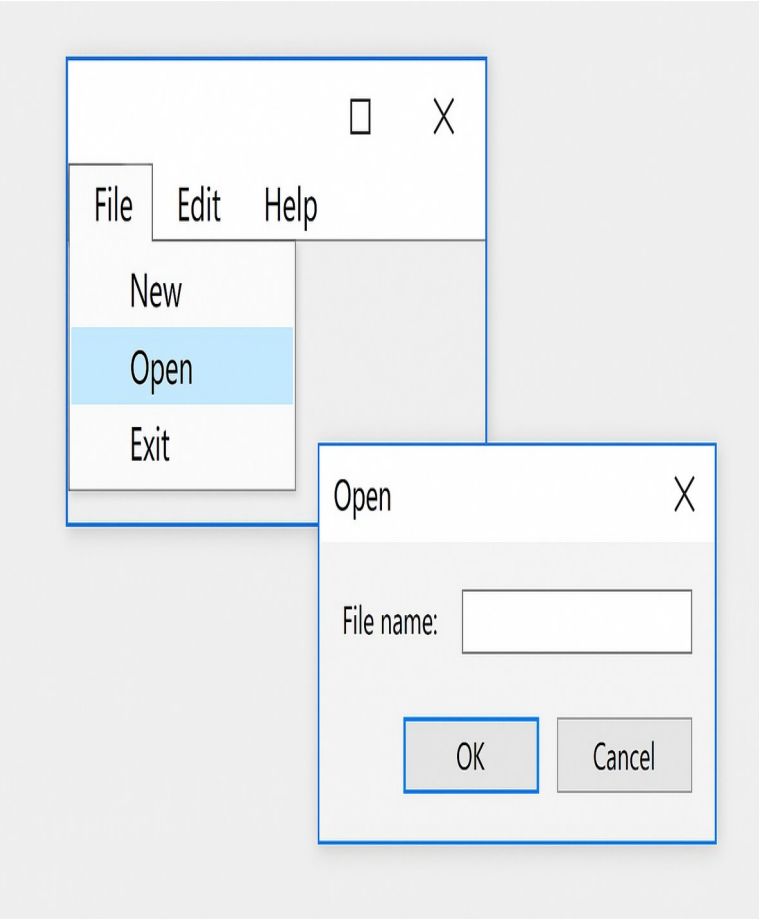
  saveFileDialog.ShowDialog();
- **ColorDialog:** Choose a color from a palette.

  colorDialog.ShowDialog();
- **FontDialog:** Select a font and style.

  fontDialog.ShowDialog();

# Dialogs

```
private void btnOpenFile_Click(object sender, EventArgs e) {
    OpenFileDialog openFile = new OpenFileDialog();
    openFile.Title = "Select a File";
    openFile.Filter = "Text Files (*.txt)|*.txt|All Files (*.*)|*.*";
    if (openFile.ShowDialog() == DialogResult.OK) {
        MessageBox.Show("Selected File: " + openFile.FileName);
    }}
```




Note:

- Menus: navigation & commands.
- Dialogs: predefined pop-up windows for user interaction.
- Both enhance usability and make applications more user-friendly.

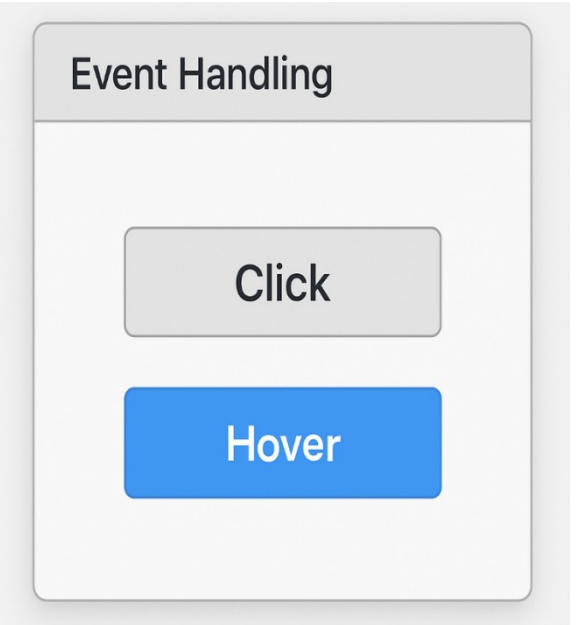
# Event Handling

- Event Handling is the mechanism in event-driven programming that allows applications to respond to user actions or system triggers.
- An event is an action. E.g., clicking a button, hovering the mouse, typing in a textbox.
- An event handler is a method that executes when the event occurs.
- Events based on the Publisher-Subscriber pattern:
  - Control (Publisher): raises an event, such as a button raising a Click event.
  - The Event Handler (Subscriber) method is attached to handle the event.

# Event Handling

```
private void btnSubmit_Click(object sender, EventArgs e) {
    MessageBox.Show("Submit button clicked!"); }
```

- btnSubmit_Click:  event handler.
- btnSubmit (Button): event source (publisher).
- Click: event.
- MessageBox.Show(...): action taken.




- **Subscribing to Events:** There are two ways to attach event handlers:
  - **Using Designer(Visual Studio):**  Drag a Button -> double-click it -> auto-generates btn_Click method.
  - **Using C# Code Programmatically:**  btnSubmit.Click += new EventHandler(btnSubmit_Click);

# Thank You!

?